Literature review

What already exists?
Where the gap is
How does our project fill in the gap?

Themes to work around

- **Task interruption/resumption**: feat 1, search for work on session resumption etc.
    1. https://arxiv.org/pdf/2001.10898 [screentrack]:

       What screentrack does. ScreenTrack helps people find and return to documents, websites, and apps they used earlier by recording the screen in the background like a time lapse video, letting the user scrub through it visually rather than search by filename or date, and click on the moment they recognize to reopen that resource. The idea is that people remember what they were doing as a sequence of scenes better than they remember exact file names or dates, so ScreenTrack works with that natural visual memory instead of against it. In their lab study it helped people find and resume things faster in some cases, but the authors are upfront about its limits: it works well for recent activity but becomes slow and tedious for anything further back, it depends on things looking visually distinct so it works for tasks like shopping or slide design but the authors admit it may not work for text heavy tasks like writing in a word processor, and it was only tested in a short controlled lab setting rather than real long term, cluttered, multi app use.

       Gap: It does not scale to medium or long-term retrieval, working only for recent tasks or when the user roughly remembers when something happened, since its video-timeline interface makes searching a long history slow and manual. It also relies on the user's own episodic memory to locate a snapshot rather than solving retrieval directly. More importantly, it was built and tested only for visually distinct tasks like shopping and slide design, and the authors state it may not generalize to text-heavy tasks like writing in a word processor. The evaluation itself was a short, controlled lab study with no real-world test of cluttered, multi-tab, multi-app setups, and the authors list long-term deployment as future work rather than something already solved.

ATREMIS: avoids this problem structurally rather than trying to fix it. Feature 1 does not use screen recording or visual scrubbing at all. It works from the file system and the user's own saved notes, meaning it does not care whether two files look alike on screen, since it is matching by filename, recency, and explicit text notes rather than pixels. It also sidesteps the "searching a long video" problem, since a workspace has a bounded scope by design, so there is a short list to show rather than an openended timeline to scrub through. The specific mechanism responsible is Feature 1 itself, combined with the workspace boundary described in Section 2.

2. https://arxiv.org/pdf/2603.29194 :

   What the paper does. Instead of keeping one flat history, the authors split the AI's memory into three layers. Working memory holds only the most recent turns, kept small. Episodic memory holds a compressed summary of each past session, and new summaries are blended in gradually so old sessions fade slowly instead of disappearing suddenly. Semantic memory holds stable facts about people, things, and events, pulled out from those episodic summaries. When the AI needs to answer something, it does not just replay the full history. It automatically decides how much to pull from each of the three memory layers depending on what is relevant to the current question. During training, they also add a rule that punishes the semantic memory for changing too suddenly between sessions, which is what keeps facts stable instead of drifting.They tested this on three benchmarks built for long multi session conversations. Compared to earlier methods, their system remembered more accurately over time, made up fewer false memories, and used less context space per question, meaning it was both more accurate and cheaper to run.

   The gap: When an AI assistant has a very long conversation across many sessions, it starts to struggle. If it tries to remember everything, the cost gets very high and old irrelevant details start confusing it. If it compresses or summarizes to save space, it starts losing facts and even makes things up that never happened, called false memories. Other researchers have tried to fix this before, but each fix only solves part of the problem. Some save space but only work within a single session, not across many sessions. Some pull back relevant memories well but still drift and hallucinate over time. Some track how well memory is retained but do not actually control drift. Some compress text to save tokens but do not keep

any real memory across sessions at all. So no one had a method that was both efficient and accurate at the same time over long multi session conversations.

ARTEMIS: does not attempt to solve this the way the paper does. It does not build a layered working, episodic, and semantic memory system with automatic decay and drift control. Instead, it constrains the problem before it starts, by scoping memory to a single explicit workspace and keeping that memory visible and editable through the What ARTEMIS Knows panel. Feature 1 again explicitly avoids AI interpretation, so there is no long freeform history for drift to happen to in the first place.

3. https://arxiv.org/pdf/2606.30306

   What it does: This is a 2026 survey of how LLM-based AI agents handle persistent memory and state, with a section specifically on coding agents. It does not propose a new system itself, it maps out the landscape. Its most directly useful point is that even coding agents, a narrower and more mature category than general personal assistants, are documented as losing coherence across sessions, repeating known mistakes and forgetting a project's established conventions.

   The Gap: The gap it surfaces is not something the survey itself is meant to fix, but it shows that the systems it surveys mostly aim persistence at code-editing conventions specifically, not at the broader mix of documents, notes, and files that a general personal knowledge worker actually deals with, and that persistence in these systems is usually an automatic background process the user cannot inspect or control.

   ARTEMIS: lines up with features 1 and 5. Artemis targets the same underlying coherence problem but for a bounded personal workspace rather than an open ended codebase, and does it through an explicit, user visible mechanism, the What Artemis Knows panel, rather than an opaque background memory process. This is a partial fill, since artemis narrows the ambition on purpose, consistent with Section 2 framing of trading full scope for something demonstrable in two semesters, rather than attempting to solve persistent AI memory in general.

4. https://www.microsoft.com/en-us/research/wp-content/uploads/2016/11/CHI_2007_Iqbal_Horvitz-1.pdf

The paper is about how people actually get disrupted from their main computing task and what helps them recover, studied through a real field study of ordinary computer use rather than a lab task. Their method was to log real workplace activity and look for patterns behind faster resumption. What they found, and what later work like ScreenTrack builds directly on, is that visible on-screen cues, meaning windows or applications left open rather than minimized, act as a strong reminder that helps people pick a suspended task back up.

The gap is that this paper only observes and diagnoses the problem. It does not propose or test any system, so there is no answer to what happens when leaving a window open is not a viable strategy anymore, for example after a reboot, a multi-day gap, or switching devices. This paper is the ancestor of the entire visual-cue line of research. ScreenTrack turned its finding into an actual tool using screen recordings, and the gaze-based dashboard work below turned it into a tool using eye-tracking.

ARTEMIS turns this same descriptive finding into a structured, durable feature. Feature 1 is effectively a stable substitute for the physical cue Iqbal and Horvitz studied, since it surfaces the equivalent information, what was recently active, what was saved, what the user said was next, as a persistent list rather than something that depends on a window physically staying open on a screen. This is a partial fill of the gap, since ARTEMIS operationalizes the finding but has not run the kind of field validation Iqbal and Horvitz did, which is exactly what our O2 objective is set up to do.

- **Bounded ai agents and trust:** search for human in loop ai agents and scoped access llm agents compare against claude gemini copilot
    1. https://arxiv.org/pdf/2512.03180

    What already exists: the authors built a framework called AGENTSAFE from IBM. It watches the agent's whole loop, plan, act, observe, reflect, and every tool it uses, and maps each one against a structured list of known risks. There are three main parts worth knowing. First, capability-scoped sandboxes, meaning every tool the agent can use gets a narrow, clearly defined box of what it is allowed to do, and this is enforced at the system level, not just suggested. Second, policy-as-code gates, meaning rules like "this agent cannot exfiltrate data" are written as actual code that checks actions in real time instead of just being written

guidelines. Third, an escalation workflow, so if an action is considered high-impact, it automatically stops and gets sent to a human for approval, and that decision is logged and cryptographically connected to what happened.

What it achieves: they demonstrate the framework on a real example of a healthcare AI agent that is restricted to read-only medical records. If it makes a high-impact suggestion, like changing a treatment plan, the action is automatically sent to a human doctor before anything can happen. So basically, it shows that these safety controls can actually be implemented and enforced through software instead of just existing as policies or guidelines.

What's missing: the main issue is that this is built for enterprise settings like healthcare systems, corporate AI platforms, and organizations with a lot of security and compliance infrastructure. It relies on things like cryptographic tracing, machine-readable risk registers, and organizational escalation workflows. It assumes that there is a whole company's security infrastructure behind the agent. It doesn't really address what this would look like for one person using an AI assistant on their own laptop, where none of that infrastructure exists and the person setting it up isn't a security engineer.

How ARTEMIS fills the gap: it doesn't try to replicate all the machinery of AGENTSAFE, it replicates the principle at a personal scale. The workspace model in Section 2 is basically a plain-language, opt-in version of capability scoping. There is no code required, the user just adds a folder instead of a system administrator defining a sandbox. The What ARTEMIS Knows panel works like a simplified audit trail that anyone can actually read, instead of requiring someone who understands a provenance graph. The approval step before any file change or outbound action is also a lightweight version of AGENTSAFE's escalation workflow. So this is only a partial fill of the gap. ARTEMIS completely strips out the cryptographic and organizational layers and trades some of AGENTSAFE's rigor for something that a normal, non-technical user can actually understand and operate.

2. **https://arxiv.org/pdf/2603.18096 [A Trace-Based Assurance Framework for Agentic AI Orchestration]**

What already exists: the authors built a system that records everything an AI agent does as a structured trace, a sequence of messages and actions, with explicit contracts attached to each step that can be automatically checked as pass or fail. If something breaks, the framework points to the exact step that violated its contract and can replay that failure deterministically to reproduce it. On the governance side, every action gets mediated as one of three outcomes: allow it, rewrite it to something safer, or block it outright, and per-agent capability limits are enforced throughout.

What it achieves: it gives developers a way to stress-test a multi-agent system before shipping it, deliberately injecting faults at the points where it talks to outside services, memory, or retrieval, and measuring how well the system contains the damage instead of letting it cascade into something worse.

What's missing: this is a developer-facing testing and evaluation tool, not something an end user ever interacts with. It's built to catch problems before deployment, not to explain a pending action to a person or get their sign-off in the moment. And critically, there's no reversal mechanism described anywhere in it, once an action is mediated and allowed to run, there's no way to undo it afterward.

How ARTEMIS fills the gap: this is one of the clearest direct matches in the whole theme. The framework's biggest blind spot, no way to reverse an action once it executes, is exactly what ARTEMIS's undo step in Section 2 is built to cover. Where this framework mediates actions using machine-checkable contracts before they run, ARTEMIS mediates them with a plain-language preview a person actually reads before they run, and then adds the reversal option this framework never has at all. This is only a partial fill in the other direction, though: ARTEMIS has no deterministic replay and no automatic contract verification, it depends entirely on a person looking at a preview and deciding, which is a weaker guarantee than a machine-checked contract, but a much more usable one for someone without a testing or engineering background.

3. **https://arxiv.org/pdf/2603.11088 [The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey]**

What already exists: this survey maps out how agentic AI systems get attacked and defended, and one section covers privilege separation as an architectural defense, the same underlying idea as least-privilege scoping,

described from the security side rather than the governance side. It compares this to how operating systems separate a trusted kernel from untrusted user programs, applied here to separating a trusted planning component from an untrusted component that processes raw, possibly manipulated, external data.

What it achieves: thorough, structured coverage of the defense landscape, but the part that matters most here is a section that directly examines the limitations of human-in-the-loop approval, the exact mechanism ARTEMIS relies on throughout Section 2. It states plainly that frequent validation prompts cause decision fatigue that undermines the safety benefit they're supposed to provide, and that most people don't have the security background to judge what they're actually being asked to approve, so in practice they either blindly approve risky things or overreact to harmless ones.

What's missing: no proposed fix. The survey names this as an open challenge the field hasn't solved, not a problem it solves itself.

How ARTEMIS fills the gap: it honestly doesn't, and that needs to be said plainly rather than smoothed over. This paper identifies a real, documented weakness in the exact approach Section 2 depends on, stopping for approval before every consequential action. The proposal currently has no mechanism to prevent the fatigue this paper describes, nothing that distinguishes a routine, low-stakes approval from a genuinely risky one, and nothing that reduces prompts as trust builds over time. This is a limitation ARTEMIS inherits from relying on human-in-the-loop approval at all, not something its bounded workspace design solves. It's exactly the kind of thing your O3 evaluation should be built to test directly, whether approval fatigue actually shows up as people use ARTEMIS over repeated sessions, rather than assuming the design avoids it by default.

- **Agentic reasoning/undo in ai**: search for reversible ai actions , undo in ai assisted systems etc.
  1-https://papers.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf

What already exists?

Existing research has shown that LLM-based agents can perform multi-step tasks by reasoning about their actions and learning from previous experiences. Reflexion extends this idea by allowing an agent to evaluate its previous attempts, generate verbal reflections, store them as episodic memories, and use them to improve future plans and actions. The approach was evaluated on decision-making, reasoning, and programming tasks and showed improvements over baseline approaches.

Where is the gap?

However, Reflexion mainly focuses on improving an agent's performance through previous task experiences. Its memory represents lessons from the agent's previous attempts rather than the ongoing context of a human user's real-world workspace. The experiments also use bounded memory contexts. Therefore, there is room to explore how agentic reasoning and memory can be used to maintain human work context and support users when they return to interrupted tasks.

How does ARTEMIS fill the gap?

ARTEMIS applies agentic reasoning in a bounded desktop workspace. Instead of focusing only on learning from failed attempts, ARTEMIS uses workspace context such as recently changed files, saved notes, and the user's intended next action to help reconstruct interrupted work. It also combines multi-step agentic planning with explicit workspace boundaries, user approval, and reversible actions.

2-https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf

What Already Exists?

Existing research has shown that Large Language Models (LLMs) can be improved by giving them access to external tools. One important example is **Toolformer**, which teaches a language model to decide **when to use a tool, which tool to use, and what information to provide to that tool**. Toolformer can use different tools such as search engines, calculators, question-answering systems, translation systems, and calendars.

This allows the model to perform tasks that are difficult to solve using its language knowledge alone.

Before Toolformer, many tool-use approaches either required **large amounts of human-labeled data** or were designed for a **specific task with a specific tool**. Toolformer introduced a more general approach in which the model learns tool use in a self-supervised way instead of depending heavily on human annotations.

However, these existing approaches mainly focus on **using individual tools** rather than managing a complete multi-step task. Toolformer can decide to call a tool, but its approach does not fully support situations where the result of one tool needs to become the input for another tool. Therefore, existing work provides an important foundation for agentic systems, but it does not completely solve the problem of **multi-step reasoning and planning with tools**.

In the context of Agentic Reasoning and Planning, this existing research is important because it demonstrates that an LLM can move beyond simply generating text and can instead **select and use external resources to achieve a goal**.

What Is the Gap?

Although existing research has demonstrated effective tool use, there are still important limitations in **multi-step reasoning, planning, and interactive tool use**.

First, Toolformer cannot properly **chain multiple tools together**. In other words, the output of one tool cannot easily be passed as the input to another tool. This is a major limitation for complex tasks because many real-world tasks require several actions in a particular order.

For example, consider a task such as:

> "Read my documents, prepare a summary, and email the summary to my supervisor."

A useful agent needs to perform several connected steps:

Read documents → extract information → create summary → prepare email → send email

Each step depends on the result of the previous step. Existing tool-use approaches such as Toolformer do not fully address this type of chained task execution.

Second, existing approaches have limited support for **interactive reasoning**. For example, when a search does not provide enough information, an agent may need to examine the results, change the query, search again, and then use the new information. Toolformer identifies this limitation itself, particularly for search-based tools.

Third, Toolformer can be sensitive to the **exact wording of the user's input** when deciding whether to call an API. This can make tool selection less reliable when users express the same goal in different ways.

Finally, existing tool-use systems do not primarily focus on **bounded and user-controlled task execution**. In real-world desktop work, users may not want an AI system to have unrestricted access to their files or to perform actions without their knowledge.

Therefore, there is a need for an agentic system that can combine:

- **Reasoning**
- **Task decomposition**
- **Multi-step planning**
- **Tool selection**
- **Sequential tool execution**
- **User approval**
- **Bounded access to user data**

This gap provides the motivation for ARTEMIS.

What Will ARTEMIS Cover?

ARTEMIS (Adaptive Reasoning, Task Execution and Multi-agent Intelligence System) will address this gap by developing a **bounded, user-controlled desktop agent** that can reason about a user's request, plan the required steps, select appropriate tools, and execute tasks while keeping the user in control.

Unlike a simple chatbot, ARTEMIS will not only provide an answer. It will determine **what needs to be done and whether the task requires multiple steps**. For example, a request to prepare and send a summary can be divided into document reading, information extraction, summary generation, email drafting, and final approval.

The system will work within an explicitly selected **workspace**. Users will choose the files, tabs, and notes that ARTEMIS is allowed to access. ARTEMIS will not automatically access the user's entire computer system.

ARTEMIS will also provide several practical capabilities, including:

- **Document understanding and synthesis** – combining information from selected documents into a new document.
- **Web search** – retrieving information from the web when the required information is not available in the workspace.
- **Code assistance** – explaining code and suggesting fixes without automatically executing them.
- **Document drafting** – creating a first draft from existing workspace material.
- **Email assistance** – preparing emails from available information and sending them only after user approval.
- **Task automation** – identifying repeated manual tasks and asking the user whether they should be automated.

A key part of ARTEMIS is its **human-in-the-loop approach**. Actions that modify files or leave the workspace, such as sending an email, will require user approval. The system will also provide an undo mechanism for supported actions. This makes the agent more controlled and reviewable rather than completely autonomous.

Therefore, ARTEMIS will extend the idea of tool-using language models toward a more complete **agentic reasoning and planning system**, where the agent can move from:

User Request → Reasoning → Task Planning → Tool Selection → Multiple Actions → User Approval → Final Result

The overall goal is to demonstrate that an agent can perform useful multi-step desktop tasks while maintaining **user control, limited access, transparency, and trust**. The ARTEMIS proposal specifically identifies agentic reasoning as the mechanism that allows a single request to be broken into the appropriate sequence of tool calls

3-https://dl.acm.org/doi/epdf/10.1145/3586183.3606763

What already exists?

Previous work such as Generative Agents has demonstrated that LLM-based agents can use memory, reflection, and planning to produce coherent behavior over time. The system stores past experiences, retrieves relevant memories, generates higher-level reflections, and uses these reflections to create future action plans. This shows that memory-based reasoning and planning can enable agents to make decisions beyond a single prompt-response interaction.

What is the gap?

However, this work primarily focuses on simulating believable human behavior in a controlled virtual environment. It does not focus on helping a real user resume interrupted work across a bounded desktop workspace, select tools for practical tasks, or require user approval before consequential actions. In addition, the paper reports failures such as incorrect memory retrieval and fabricated memory details.

What will ARTEMIS cover?

ARTEMIS will apply agentic reasoning and planning to a bounded desktop workspace. Given a user request, the agent will determine what information is needed, retrieve relevant workspace context, select appropriate tools, and plan a sequence of actions. Unlike a fully autonomous agent, ARTEMIS will require user approval before actions that modify files or leave the workspace, while providing preview, audit, and undo mechanisms.

4-https://arxiv.org/pdf/2210.03629

What Already Exists

Previous research has shown that Large Language Models can perform reasoning and planning for task solving. Chain-of-Thought approaches allow LLMs to reason step-by-step, while other approaches allow language models to select and perform actions in interactive environments. ReAct combines these two capabilities by interleaving reasoning, actions, and observations. This allows an agent to use reasoning to decide what action to take, observe the result, and then update its next action or plan. ReAct was evaluated on question answering, fact verification, interactive environments, and web navigation, showing that combining reasoning and acting can improve task performance and make the decision process more interpretable.

What is the Gap?

Although ReAct demonstrates that reasoning and acting can work together, there are still limitations for practical desktop-based agentic systems. ReAct can experience reasoning errors, repetitive action loops, and difficulties when an action produces uninformative results. The paper also notes that complex tasks with large action spaces

may require more demonstrations than can easily fit within the model's context. Another gap is that existing approaches mainly focus on solving individual tasks in specific environments rather than maintaining a bounded workspace context for users across everyday activities. ARTEMIS addresses a practical scenario in which users work with multiple files, notes, web information, documents, code, and communication tasks and may need to resume work after an interruption. The ARTEMIS proposal identifies this need and focuses on a small, explicitly selected workspace rather than unrestricted system access.

What Will ARTEMIS Cover?

ARTEMIS will extend the idea of agentic reasoning and planning into a bounded desktop workspace. The agent will understand a user's request, decompose it into multiple steps, select appropriate tools, execute the required actions, observe the results, and re-plan when necessary. For example, a request to prepare a summary and email it can be broken into reading documents, synthesizing information, drafting the email, and sending it after user approval.

ARTEMIS will also focus on controlled and trustworthy autonomy. The agent will only access information explicitly added to a workspace, while actions that modify files or leave the workspace will require user approval. Preview, approval, audit, and undo mechanisms will keep the agent's planning and execution bounded and reviewable.

Therefore, the main contribution of ARTEMIS is not simply adding another reasoning method. It is applying agentic reasoning and multi-step planning to a practical desktop workspace while combining tool selection, task decomposition, context awareness, bounded access, human approval, and reversible actions.

5-https://arxiv.org/pdf/2305.10601

1. What Already Exists?

Existing research in **Agentic Reasoning and Planning** has already focused on making Large Language Models (LLMs) better at solving problems by allowing them to reason through a task instead of directly giving one answer. Earlier approaches such as **Chain-of-Thought (CoT)** allow an LLM to break a problem into smaller reasoning steps. However, CoT generally follows one reasoning path from beginning to end. If an early decision is wrong, the model may continue using that wrong direction instead of trying another option. The **Tree of Thoughts (ToT)** approach was introduced to improve this process. Instead of following only one reasoning path, ToT allows the model to generate multiple possible thoughts or solutions, evaluate them, and continue with the

more promising one. It can also look ahead and go back to an earlier step when the current path is not useful. In this way, ToT brings ideas such as **search, planning, evaluation, and backtracking** into LLM reasoning.

This means that a lot of important work already exists for **making LLMs reason and plan better**. ToT, for example, represents a problem as a tree where different branches represent different possible reasoning paths. The model generates possible next steps, evaluates those steps, and uses search methods such as Breadth-First Search (BFS) or Depth-First Search (DFS) to explore the possible solutions. This is an important development for agentic reasoning because the model is no longer simply generating the next word or next answer; it is considering possible future steps before deciding what to do. The paper also discusses other work related to planning, decision-making, self-reflection, and search, showing that researchers have already started combining LLM reasoning with traditional planning and search techniques.

However, these existing approaches are mainly focused on **solving reasoning and search-based problems**. For example, ToT was evaluated on tasks such as Game of 24, creative writing, and Mini Crosswords. These experiments showed that allowing the model to explore different reasoning paths can improve performance. In Game of 24, for example, the paper reports a large improvement when ToT was used compared with standard Chain-of-Thought reasoning. Therefore, we can say that **existing research has already established the importance of multi-step reasoning, planning, search, evaluation, and backtracking for LLMs**.

2. What Is the Gap?

The main gap is that existing research has made strong progress in **reasoning and planning inside the model**, but there is still a need to apply these ideas to **practical, real-world, multi-step tasks** where an AI agent has to understand a user's goal, decide what actions are needed, use different tools, and complete the task while keeping the user in control. The Tree of Thoughts paper itself explains that its work mainly studies reasoning and search problems and suggests that more complex real-world applications, such as coding, data analysis, and robotics, create opportunities for further research into better planning and reasoning. It also points out that search-based reasoning can require more resources and that the evaluation heuristics used by the model are not always perfect.

Another important gap is that **reasoning alone is not enough for a real assistant**. A real desktop assistant needs to do more than think about possible solutions. It may need to read files, combine information from different documents, search for missing

information, prepare a document, suggest a code change, organize files, or prepare an email. These tasks require the agent to connect its reasoning with **actual actions and tools**. At the same time, giving an AI unrestricted access to a user's computer creates trust and safety problems. Your ARTEMIS proposal identifies this problem directly: existing assistants can either require the user to give an explicit command every time or can operate with broad access to the user's system. The proposal identifies a need for an assistant that can help with multi-step work without giving it unlimited access or removing the user's control.

There is also a gap in **safe and controlled autonomy**. An agent that can independently plan and perform actions can be useful, but it can also make mistakes. For example, automatically changing a file or sending an email can cause a problem if the agent misunderstands the user's intention. Therefore, there is a need for an approach where the agent can reason and plan independently to some extent, but important actions still require human approval. ARTEMIS addresses this issue by using a bounded workspace and an approval-based interaction model. The system only accesses information that the user has explicitly added to the workspace, and actions that change files or leave the workspace require the user to review and approve them.

So, **in very simple words**, the gap is this: existing research mainly asks **"How can we make an LLM think and plan better?"** ARTEMIS goes one step further and asks **"How can we use this reasoning and planning to perform real multi-step desktop tasks while keeping the user's data, control, and approval protected?" This is the main research direction that connects your literature review to ARTEMIS.**

What Will ARTEMIS Cover?

ARTEMIS will cover agentic reasoning and planning for practical desktop work. Instead of working like a normal chatbot that simply gives an answer, ARTEMIS will receive a user's goal, understand what needs to be done, decide which tools are needed, and determine whether the task requires multiple steps. For example, if a user asks ARTEMIS to prepare a summary from several documents and then email that summary to a supervisor, ARTEMIS can break the request into smaller actions. It can identify the relevant documents, collect the required information, create the summary, prepare an email, and then ask the user for approval before sending it. This is the agentic part of ARTEMIS because the system is planning a sequence of actions instead of only producing one response.

ARTEMIS will also cover tool selection and multi-step task execution. The agent will decide which available tool is appropriate for the current step. For example, it

may need to read a file first, then combine information from several files, then create a new document, or search the web if the required information is not available inside the workspace. The proposal specifically describes features for document synthesis, web search, mailing, document drafting, code assistance, and repeated-task automation. These features give ARTEMIS a practical environment in which agentic reasoning can be used instead of testing reasoning only on artificial problem-solving tasks.

A major part of ARTEMIS will be bounded and human-controlled planning. ARTEMIS will not have unrestricted access to the user's entire computer. The user first selects a workspace, and ARTEMIS only works with the files, tabs, and notes that are explicitly added to that workspace. The system will also show what information it currently knows through the "What ARTEMIS Knows" panel. This makes the agent's knowledge visible to the user. Furthermore, when ARTEMIS wants to perform an action that changes a file or communicates outside the workspace, it will show the action to the user first. The user can approve it, and actions can be undone where applicable.

Therefore, ARTEMIS will not try to build a completely autonomous AI that can control everything on a computer. Instead, it will focus on bounded autonomy. The agent will be able to reason, plan, select tools, and prepare actions, but the user will remain in control of important decisions. This is especially important for tasks such as file organization, code changes, automation, and sending emails. The proposal also plans to evaluate whether this approach improves task completion time, cognitive workload, and user trust.

- **Voice interaction parity**: feat 9, serach for speed vs text interface task completion
  1-https://www.mdpi.com/2078-2489/15/9/579?utm_source=chatgpt.com

Existing Work, Research Gap, and ARTEMIS

Voice User Interfaces (VUIs) are already widely studied in Human-Computer Interaction. Previous research has examined the usability, user experience, accessibility, personalization, privacy, and technical challenges of voice assistants. Researchers have also studied how different groups of users interact with voice assistants, including older adults, people with disabilities, native and non-native English speakers, and users from different cultural backgrounds. These studies show that voice interaction can provide convenient and hands-free interaction, but it can also create problems such as speech recognition errors, misunderstanding of commands, poor responses, and difficulties in recovering from errors.

An important issue identified in previous research is that voice assistants may not perform equally well for all users. Differences related to age, ethnicity, dialect, language background, and culture can affect the user's experience and system performance. Research has therefore identified the reduction of performance disparities across different demographic groups as an important future research challenge. In addition, the literature still lacks standardized and context-aware methods for evaluating voice user experience and performance. This makes it difficult to consistently determine whether a voice system provides an equally good interaction experience for different users and situations.

ARTEMIS addresses the voice-interaction problem by integrating speech input and output into a bounded agentic desktop assistant. Instead of typing a request, users can speak to ARTEMIS, and the system can provide responses through speech. Voice interaction is not treated as a separate chatbot; it is an alternative way to access other ARTEMIS features, such as document-related tasks, web search, code assistance, and email drafting. The system also keeps the user in control by maintaining the same approval requirements for actions performed through voice interaction.

For evaluation, the current ARTEMIS proposal focuses on comparing speech-based interaction with typed interaction. The evaluation will examine whether voice interaction affects ease of use and task completion time. The broader ARTEMIS evaluation also considers cognitive workload and user trust. Therefore, ARTEMIS contributes by studying voice interaction in a real desktop productivity environment rather than treating voice only as a question-and-answer interface.

2-https://pmc.ncbi.nlm.nih.gov/articles/PMC9063617/?utm_source=chatgpt.com

1. What Already Exists in Previous Research?

Answer:

Previous research has already studied **voice assistants and their usability** in many different ways. The research paper we are studying is itself a **systematic literature review**. It reviewed previous studies to understand what researchers have already investigated about voice-assistant usability. The review finally included **29 relevant studies** from ACM and IEEE databases.

The existing research mainly focuses on **how users interact with voice assistants and what factors affect their experience**.

Researchers have studied different categories of factors.

First: Voice-related factors

Researchers have studied characteristics of the assistant's voice, such as:

- voice gender,
- voice accent,
- voice personality,
- and different voice styles.

For example, studies have compared male and female voices and different accents to understand their effects on users' trust, effectiveness, and interaction.

Second: User-related factors

Researchers have also studied characteristics of the **user**, such as:

- user gender,
- personality,
- previous experience,
- and language/accent.

This shows that the same voice assistant may not provide exactly the same experience to every user.

Third: Task-related factors

Previous studies have investigated the type of task users perform with a voice assistant.

For example, research has examined:

- interactive tasks,
- drawing tasks,
- driving simulations,
- auditory tasks,
- and speech-to-text tasks.

So, researchers have already understood that **the type and context of a task can affect voice-assistant usability**.

Fourth: Conversational style

Researchers have also studied **how the assistant communicates with the user**.

For example:

- Does it ask a clarifying question?
- Does it respond empathetically?

- Does it use conversational fillers such as “um” or “uh”?
- Does its communication style match the user's communication style?

These factors have been shown to influence engagement, efficiency, and attitude toward the voice assistant.

Fifth: Human-like behavior

Previous research has also studied **anthropomorphism**, which means making the voice assistant appear more human-like.

Researchers have investigated:

- personalization,
- personification,
- smart speakers,
- robots,
- human-like voices,
- gaze,
- and other human-like behaviors.

Some studies found that personalized or more human-like assistants can increase trust and engagement.

Sixth: Usability measures

Previous studies have also measured usability.

The paper identifies the traditional ISO 9241-11 measures:

- **Effectiveness** – Can the user successfully complete the task?
- **Efficiency** – How much time/effort is required?
- **Satisfaction** – Is the user satisfied with the interaction?

But voice-assistant research also measures additional factors such as:

- **Trust**
- **Likeability**
- **Acceptance**
- **Perceived intelligence**
- **Perceived humanness**
- **Social presence**
- **Mental workload**

- **Cognitive load**

What Is the Research Gap?

Answer:

The research gap is **not that voice assistants have never been studied**.

This is very important.

There is already a large amount of research on voice assistants. The real gap is that the existing research is **fragmented** and does not completely answer how voice interaction should be evaluated when it is used for broader, practical computer work.

The paper itself identifies several gaps.

Gap 1: Existing research studies different factors separately

Previous researchers have studied many individual factors, such as:

accent → trust
gender → usability
personality → social presence
conversational style → engagement
anthropomorphism → trust

But the relationships between many variables and usability measures are still incomplete.

The paper specifically says that some relationships are well studied, while others are still unclear or have very few studies.

For example, the paper says that **accent** has been studied in relation to effectiveness, but it remains unclear whether the effect is the same for users of different genders. Similarly, the relationship between **gender and efficiency** has received very little research.

Simple meaning:

Researchers have studied **many pieces of the puzzle**, but there are still missing connections between those pieces.

Gap 2: Voice-specific usability is not fully covered by traditional usability frameworks

The ISO 9241-11 framework mainly focuses on:

Effectiveness + Efficiency + Satisfaction

But the paper found that voice assistants also involve:

Trust + Attitude + Machine voice/Human-likeness + Cognitive load

These factors are not completely represented by the traditional framework.

The paper therefore concludes that ISO 9241-11 alone is not enough to explain all aspects of modern voice-assistant usability.

Simple meaning:

For a normal software application, we may ask:

> “Did the user complete the task?”

But for a voice assistant we also need to ask:

> “Did the user understand what to say?”
> “Did the user have to think too much?”
> “Did the user trust the assistant?”
> “Did the user feel comfortable talking to it?”

These are especially important in voice interaction.

Gap 3: Many studies use controlled environments

Another important gap is the **research environment**.

The paper explains that many experiments were performed in controlled environments. A controlled experiment is useful for testing cause and effect, but its results may not always represent what happens in real life.

The paper specifically recommends more research in **natural settings** and more comparison between controlled and real-world environments.

Simple example:

In a laboratory, a user may be asked:

> "Ask the voice assistant to find some information."

But in real work, a user may need to:

> Find a file → read information → summarize it → create a document → search for missing information → prepare an email.

This is much more complex.

Gap 4: There is limited research on voice as a complete interaction method for desktop work

This is the part that is **most relevant to ARTEMIS**.

The paper reviews many voice-assistant tasks, but it does not establish that voice can provide **parity with typing across a complete desktop workflow**.

In other words, existing research tells us a lot about:

"How usable is this voice assistant?"

But there is still room to investigate:

> **“Can users perform the same practical computer tasks through voice as they can through typing?”**

This is where your **Voice Interaction Parity** research can fit.

You should be careful with your wording, though. Don't claim:

> “No research has ever compared voice and typing.”

Your paper does not prove that.

A safer claim is:

> **“The reviewed literature does not sufficiently address voice interaction parity across complete, multi-step desktop workflows.”**

That is a much stronger and academically safer research gap.

Gap 5: The paper itself says many aspects are still under-researched

The paper concludes that although many usability measurements have already been studied, **many aspects and relationships remain insufficiently researched**. It also states that current usability measures do not completely capture the needs of modern voice assistants.

So the overall gap is:

> **Existing research has studied voice assistants from many individual perspectives, but there is still a need for broader evaluation of voice interaction in realistic, multi-step tasks, especially when comparing voice with another interaction method such as typing.**

---

3. What Will ARTEMIS Cover?

Answer:

ARTEMIS will address this research area by providing **speech input and speech output as an alternative way of interacting with a desktop AI assistant**.

This is clearly stated in your ARTEMIS proposal.

The proposal says that the speech feature allows users to:

speak a request instead of typing it

and

hear the response instead of reading it.

Most importantly, voice is **not a separate capability**. It is an alternative interaction method for the other ARTEMIS features.

This is very important for your research.

ARTEMIS will cover Voice vs Typed Interaction

ARTEMIS will specifically evaluate whether speech-based interaction changes:

1. Ease of use

You will investigate whether users find voice interaction easy to use compared with typing.

For example:

Typing:

User types: “Summarize these documents.”

Voice:

User says: “Summarize these documents.”

Then you can compare how users experience both methods.

---

2. Task Completion Time

ARTEMIS will evaluate whether voice changes the amount of time required to complete a task.

Your proposal specifically states that the speech-based mode will be evaluated to see whether it changes **ease of use or task completion time relative to typed interaction**.

So your research can ask:

> **Is voice faster, slower, or similar to typing for the same task?**

---

3. Cognitive Workload

Your ARTEMIS proposal also includes **cognitive workload** in its evaluation.

This is important because the literature review identifies cognitive load as a voice-specific usability issue. The paper defines cognitive load as the mental capacity a person needs to communicate successfully with a voice assistant.

ARTEMIS can therefore investigate:

> Does speaking a command require more or less mental effort than typing the same command?

---

4. User Trust

ARTEMIS will also evaluate **user trust**.

This is especially important because your system is not designed as a completely autonomous assistant.

ARTEMIS uses a **bounded workspace**.

The user chooses which folder becomes the workspace, and ARTEMIS only works with explicitly shared files, tabs, and notes. It does not automatically access everything on the user's computer.

---

5. User Approval and Control

ARTEMIS also introduces:

Preview → Approve → Undo

For actions that change files or leave the application, ARTEMIS stops and asks the user for approval.

This means the system is **agentic but bounded**.

The AI can reason about what needs to be done and plan multiple steps, but it does not have unlimited freedom to act.

This is particularly relevant to trust and user control.

---

6. ARTEMIS Will Use Voice Across Multiple Tasks

This is another important difference.

ARTEMIS's voice feature can be used with its other features, including:

- resuming previous work,
- organizing files,
- combining notes from documents,
- detecting repeated tasks,
- code assistance,
- web search,
- email drafting,
- document drafting.

The proposal describes these as the main ARTEMIS features, with speech input/output acting as an alternative way to interact with them.

So ARTEMIS is not only testing:

“Can the system understand my voice?”

It is closer to testing:

“Can I use voice to perform useful computer work?”

3-https://dl.acm.org/doi/10.1145/3772318.3791747

1. What Already Exists?

There is already a lot of research on **Voice User Interfaces (VUIs)** and **Voice Assistants (VAs)**.

First, existing VUI research evaluates voice assistants using **objective performance measures**. These include things such as **task success, task completion time, number of errors, and number of turns needed to complete a task**. These measures tell researchers whether a voice assistant can successfully complete a task and how efficiently it does so.

Second, researchers also use **questionnaires and self-report methods** to understand the user's experience. The paper mentions commonly used methods such as **System Usability Scale (SUS), AttrakDiff, and User Experience Questionnaire (UEQ/UEQ+)**. These methods measure things such as usability, satisfaction, attractiveness, and other aspects of UX.

Third, previous research has already investigated **speech as a source of information about the user's state**. Human speech contains more than just words. It also contains vocal information such as:

- pitch,
- speech rate,
- pauses,
- intensity,
- jitter,
- shimmer,
- voice quality,
- filled pauses,
- repetitions, and
- self-repairs.

Previous studies have connected these speech characteristics with things such as **mental effort, frustration, engagement, uncertainty, and vocal tension**.

There is also existing research connecting speech characteristics with important UX concepts such as **satisfaction, trust, and attractiveness**. This means that researchers already know that a person's way of speaking can provide information about how they are experiencing a system.

Existing VUI research has also studied different **system behaviours**, such as:

- response latency,
- error handling,
- confirmation behaviour,
- repair strategies,
- personality,
- conversational tone, and
- voice quality.

For example, the paper explains that response delays can affect trust and engagement, while good confirmation and error-repair strategies can improve the user's feeling of control.

So, in **very simple words**:

> **What already exists is research on voice assistants, their usability and performance, questionnaires for measuring UX, and research showing that speech characteristics can provide information about a user's emotions, effort, frustration, engagement, trust, and satisfaction.**

---

2. What Is the Gap?

Even though all this research already exists, the paper identifies an important gap.

Gap 1: Most UX evaluation happens after the interaction

The paper says that VUI evaluation still mainly depends on **post-hoc questionnaires and general performance measures**.

The problem is that these methods ask users to remember their experience **after the interaction has finished**.

For example, a user might become frustrated at one particular moment because the assistant did not understand them. Later, when they fill in a questionnaire, they may not remember that exact moment clearly.

The paper explains that self-reports can be affected by:

- **recency bias**,
- **politeness/social desirability**, and
- **limited self-awareness**.

Questionnaires and interviews can also interrupt the natural interaction and make users more aware that they are being evaluated.

> **In simple words:**
>
> Researchers usually ask the user **afterwards**: "How was your experience?" But this may not accurately show what the user was feeling **during the actual interaction**.

Gap 2: Not enough real-time, turn-by-turn speech analysis

This is the **main gap** highlighted by your paper.

Previous research has studied speech, but much of the available evidence comes from **post-task interviews** rather than speech recorded **during the actual VUI task**.

The paper specifically says that there are few evaluations that capture **in-situ, turn-by-turn speech during actual VUI tasks**.

This is important because researchers want to know what happens **moment by moment**.

For example:

User asks something → VA responds badly → user pauses/repeats/changes their voice → system responds again.

Researchers need to connect the user's speech at that exact moment with what happened in the system.

But existing research has not provided enough evidence for this connection.

Gap 3: It is not clear which speech features are most reliable for UX

The paper also says that there is limited guidance about **which speech features are reliable indicators of UX**.

There are many possible features:

- pitch,
- speech rate,
- pause duration,
- jitter,
- shimmer,
- HNR,
- spectral features,
- disfluencies,
- repair behaviour, etc.

But the research gap is to understand **which of these features actually correspond to particular UX dimensions** such as:

- attractiveness,
- trust,
- satisfaction, and
- naturalness.

In simple words:

We know speech contains useful information, but we still need stronger evidence about **which specific speech signals tell us that the user's experience is good, neutral, or bad**.

Gap 4: Limited connection between speech and automatic UX classification

Another gap is whether speech features can be used with **AI/ML methods** to automatically identify different levels of UX.

The paper defines three UX levels:

- **Positive UX**
- **Neutral UX**
- **Negative UX**

It investigates whether speech features can help distinguish these levels automatically.

This is important because the ultimate idea is not only to ask users about their experience. The system could potentially understand UX from the user's voice **while the interaction is happening**.

Gap 5: System behaviour and natural speech are not sufficiently connected

The paper also points out that many previous studies manipulate only one design factor, use highly scripted interactions, or depend mainly on post-hoc surveys.

Because of this, there is still a gap in understanding:

> **How do changes in system behaviour appear in the user's natural spoken behaviour during real-time interaction?**

For example, how does a long response delay, poor error handling, or poor voice quality affect the way the user speaks?

So the main gap in ONE simple paragraph is:

> **The main research gap is that although VUIs, UX questionnaires, and speech-based user-state analysis already exist, there is limited research that uses turn-by-turn speech recorded during real VUI tasks to understand the user's experience in real time. It is also not fully**

clear which speech features reliably represent UX dimensions such as trust, attractiveness, and satisfaction, or whether these features can be used by AI/ML to automatically identify positive, neutral, and negative UX.

---

3. What Will ARTEMIS Cover?

Now we have to connect this literature gap with **your ARTEMIS project**.

According to your ARTEMIS proposal, ARTEMIS is designed as a **bounded, opt-in desktop assistant**. Its purpose is not to have unlimited access to the user's computer. Instead, the user selects a folder and creates a **workspace**. ARTEMIS only works with the files, tabs, and notes that the user explicitly adds to that workspace.

The project will cover several practical tasks.

1. Resume interrupted work

ARTEMIS will help users **pick up where they left off**.

It will show:

- recently changed files,
- saved notes, and
- what the user said they were going to do next.

This is intended to reduce the time users spend reconstructing their work after taking a break.

---

2. Organize files

ARTEMIS will help users organize a messy folder.

It will read file names and short text extracts and **suggest groups**.

However, it will not automatically move the files.

The user will first see a preview and approve the action. An undo option will also be available.

---

3. Combine information from documents

ARTEMIS will be able to take information from a **small, explicitly selected set of documents** and combine important points or citations into a new file.

Existing files will not be overwritten.

---

4. Detect repeated tasks

If ARTEMIS notices that the same manual pattern has happened **three or more times**, it will ask the user whether they want to automate it.

It will **not automatically decide to automate the task**.

Every automatic run can also be undone.

---

5. Code assistant

ARTEMIS will work with source code that is already inside the workspace.

It can:

- answer questions about the code,
- explain errors, and
- suggest a fix.

The suggested fix will be shown as a **diff**, and the user must approve it before it is applied. ARTEMIS will not independently write or run code.

---

6. Web search

If the information is not available inside the workspace, ARTEMIS can perform a web search **when the user explicitly asks for it**.

It is not designed for continuous or background browsing.

---

7. Email

ARTEMIS can create an email draft using information already available in the workspace.

The user sees the complete email before it is sent.

The email is only sent **after user approval**.

---

8. Document drafting

ARTEMIS can create a first draft of things such as:

- reports,
- summaries, and
- cover letters.

The draft is saved as a new file so that the user can edit it. It is not automatically finalized or sent.

---

9. Speech input and output

This is particularly important for your **voice interaction** research.

ARTEMIS will allow the user to:

- **speak a request instead of typing**, and
- **hear the response instead of reading it**.

The proposal clearly says that speech is an **alternative way to interact with the other ARTEMIS features**, rather than a completely separate capability.

Most importantly, the same approval process will still apply when the user interacts through speech.

The research objective specifically says ARTEMIS will evaluate whether speech-based input and output changes **ease of use or task completion time compared with typed interaction**.

4-https://dl.acm.org/doi/10.1145/3706598.3713090

1. What Already Exists?

The research paper explains that **Voice User Interfaces (VUIs) and voice assistants already exist and are widely used**. Examples mentioned in the paper include voice assistants such as Siri, Alexa, and ChatGPT-powered chatbots. These systems allow users to get information, control devices, and communicate without using a keyboard or touchscreen. However, when these systems need some time to process a request, users have to wait. The paper explains that waiting can negatively affect the user experience because users may become frustrated, dissatisfied, or feel that the system is not responding.

The paper also explains that **waiting indicators already exist in graphical user interfaces (GUIs)**. For example, GUIs use progress bars, spinners, countdowns, animations, and other visual indicators to tell users that the system is processing something. These indicators can make the waiting time feel shorter, attract the user's attention, and improve pleasure and satisfaction. However, these visual techniques cannot be directly applied to VUIs because voice interfaces generally do not have a visual display.

For VUIs, some **auditory indicators already exist**. Previous work discussed in the paper uses non-speech sounds such as **beeps, ticking sounds, earcons, auditory icons, ambient sounds, and synthetic tones**. These sounds can provide feedback while the system is loading and can influence how users perceive the waiting time. So, the existing research has already established that audio feedback can be useful during waiting.

The paper also says that **human speech is already used in general voice-assistant interaction** and has several benefits. Human speech can make interactions feel more natural, understandable, trustworthy, and socially connected. Previous research discussed in the paper has shown that human speech can improve social presence, trust, engagement, and compliance in voice-assistant interactions. Explanatory speech can also help users understand what the system is doing, while humorous speech can improve perceptions of the voice assistant's social qualities.

So, in simple words, **what already exists is: voice assistants, VUIs, visual waiting indicators in GUIs, non-speech auditory indicators in VUIs, and human speech for normal voice-assistant interaction.** The important point is that these things exist separately, but their specific use during the waiting/loading period of a VUI had not been sufficiently explored.

---

2. What Is the Research Gap?

The main research gap identified by the paper is that **human speech had not been properly studied as a waiting/loading indicator in Voice User Interfaces**. The paper specifically states that previous research had explored non-speech sounds such as beeps and ticking sounds, but these sounds provide only limited and abstract information. At the same time, human speech was known to improve general interaction with voice assistants, but its use specifically during loading or waiting time remained underexplored.

Another important gap is that **most previous waiting-experience research focused on GUIs rather than VUIs**. In a GUI, a user can see a progress bar or spinner and understand that the system is working. In a VUI, the user usually cannot see such visual feedback. Therefore, users may not know whether the voice assistant is processing their request, why they are waiting, or whether the system is still working. The paper describes these delays in VUIs as relatively “invisible” and says that the waiting experience in VUIs has received less attention.

The paper then identifies a more specific gap: **there was not enough research on how different types of human speech could be designed for this waiting period**. In particular, the researchers wanted to investigate two speech characteristics: **explanation and humor**. Explanation can tell the user what the system is doing or why the user has to wait. Humor can make the waiting period more interesting and reduce boredom or annoyance. The paper says that the combination of these two elements in speech indicators for VUI waiting had not previously been properly investigated.

The researchers therefore conducted focus groups and an experiment to investigate this gap. Their focus groups with 35 experienced voice-assistant users identified **explanation and humor as the two most important design requirements**. They then tested different speech indicators with 30 participants during short and long loading times.

The findings showed that the appropriate speech design depends on the length of the wait. For **long waiting times**, a speech indicator containing both **high explanation and high humor (HH)** performed best overall. For **short waiting times**, either explanation or humor alone could be effective, because too much information during a short wait could create additional cognitive load.

So, the gap can be stated very simply:

The existing research knew that voice assistants need feedback during waiting and that audio and human speech can improve interaction, but it had not sufficiently explored how human speech—especially explanation and humor—could be deliberately designed and used as a waiting indicator in VUIs.

---

3. What Will ARTEMIS Cover?

According to your ARTEMIS proposal, **ARTEMIS is not mainly designed to solve the exact waiting-indicator problem studied in this paper**. Instead, ARTEMIS is a **bounded, opt-in desktop assistant** designed to help users manage work inside an explicitly selected workspace. The user selects a folder as a workspace, and ARTEMIS only works with files, tabs, and notes that the user has explicitly added. It does not read outside the selected workspace.

ARTEMIS will cover several practical tasks. It will help the user **resume interrupted work**, organize a messy folder, combine information from selected documents, detect repeated manual tasks, understand code and suggest fixes, search the web when explicitly requested, draft emails, create document drafts, and provide speech input and output. These features are designed to reduce the time and effort users spend on routine work.

For your research connection with **voice interaction**, the most relevant ARTEMIS feature is **Feature 9: Speech Input and Output**. This feature will allow the user to **speak a request instead of typing it and hear the response instead of reading it**. Importantly, the proposal says that speech is an alternative way to interact with the other ARTEMIS features rather than a completely separate capability. It will follow the same approval process as other interactions.

ARTEMIS will also cover **user control and trust**, which is a major part of your project design. The system will not automatically perform actions that change files or leave the application. The user will see the proposed action first and approve it. The proposal also includes an **undo mechanism**, and the “What ARTEMIS Knows” panel will show what

information the system currently has. This is intended to make the assistant bounded, transparent, and easier for users to trust.

ARTEMIS is also described as an **agentic AI system**. Instead of simply responding to one prompt, it can reason about what a request requires, choose the appropriate tools, and perform multiple steps. However, it is not intended to be fully autonomous: actions that modify files or go outside the workspace stop for user approval.

Therefore, **ARTEMIS will cover the practical use of speech interaction as part of a bounded desktop AI assistant**, while maintaining user control through approval and undo. Its research objectives specifically include evaluating whether speech-based input and output changes **ease of use or task-completion time compared with typed interaction**, as well as evaluating task completion time, cognitive workload, and user trust.

5-https://dl.acm.org/doi/10.1145/3719160.3736636

1. What Already Exists?

The first paper explains that intelligent virtual agents and conversational AI already use technologies such as **Automatic Speech Recognition (ASR), Large Language Models (LLMs), and Text-to-Speech (TTS)** to support voice-based conversations. These systems can understand a user's speech, process it through an LLM, and then produce a spoken response. Modern conversational systems have moved beyond simple fixed or scripted responses and can support more natural, free-form conversations. However, these systems can take time to respond because ASR, LLM processing, and TTS require computational resources. The paper explains that when these processes are performed locally, they can compete for system resources, while cloud-based systems can experience network congestion, connection problems, and system downtime. Therefore, **response latency is already a known problem in voice-based conversational systems**.

The paper also shows that different methods have already been used to make waiting for a response feel less uncomfortable. In traditional 2D interfaces, systems use **loading indicators, progress bars, animations, and other visual feedback** to tell users that the system is processing something. Text-based chat systems also use **typing indicators and loading feedback**. The paper specifically discusses systems such as ChatGPT, Claude, Gemini, Meta AI, and Perplexity, which use different visual indicators while generating responses. These techniques can help users understand that the system is working rather than being completely silent.

For embodied conversational agents, the existing research has also explored **conversational fillers**. These fillers can be verbal expressions such as “uh,” “uhm,” or “let me think,” combined with gestures such as touching the chin or moving the head. The purpose is to make the waiting period feel more like a natural human conversation. Earlier studies found that these fillers could reduce some negative effects of response delay. However, the previous studies discussed in the paper mainly used **scripted questions and answers, Wizard-of-Oz (WoZ) setups, or pre-recorded conversations** rather than completely free-form, real-time conversations with LLM-powered agents.

The first paper itself therefore extends what already exists by conducting a study with **truly interactive LLM-powered virtual agents**, where participants could have free-form voice conversations. The researchers tested three response delays—**1.5 seconds, 4 seconds, and 6.5 seconds**—and three conditions: **no filler, artificial wait indicator, and natural conversational filler**. Their results showed that longer delays negatively affected users' perception of the agents, while natural conversational fillers improved perceived response time particularly at 4 and 6.5 seconds.

---

2. What Is the Gap?

The main gap identified in the first paper is that previous research had already studied **response latency and ways to reduce its negative effects**, but much of that research was not conducted with **fully interactive, free-form LLM-based conversational agents**. Earlier studies commonly depended on predetermined questions and answers, Wizard-of-Oz methods, or pre-recorded conversations. Because of this, their findings were limited in terms of how well they represented real-time interaction with modern conversational agents. The authors specifically state that this limitation makes it difficult to generalize those findings to truly interactive systems.

Another important gap is that simply showing the user that the system is processing something does **not necessarily solve the user-experience problem**. The study tested artificial waiting indicators consisting of a loading icon and sound. Although these indicators communicated that the system was processing, they did not significantly improve perceived response time or the broader user-experience measures compared with having no filler. In contrast, natural conversational fillers significantly improved perceived response time at the 4-second and 6.5-second delays. However, even natural fillers did not significantly improve all broader aspects of user experience, such as engagement, good impression, competence, discomfort, or willingness to interact again.

The paper also identifies a continuing problem with **latency itself**. The study found that response delays above 4 seconds significantly degraded the user experience. Higher latency reduced engagement, good impression, perceived competence, and willingness to interact again, while increasing discomfort. The authors therefore recommend keeping response latency below 4 seconds where possible. But they also explain that technical improvements cannot completely remove the problem because network instability, hardware limitations, and increasingly complex reasoning can continue to introduce latency.

So, in very simple words, the gap is:

Existing research knows that voice-based AI can respond slowly, and it has tested loading indicators and conversational fillers. However, response delay is still a problem, and existing solutions do not completely solve it. Natural fillers can make waiting feel better, but they do not remove the actual delay or solve every aspect of the user experience.

---

3. What Will ARTEMIS Cover?

According to your ARTEMIS proposal, ARTEMIS is designed as a **bounded, opt-in desktop assistant** rather than an assistant with unlimited access to the user's whole computer. The user selects a folder and adds it as a workspace. ARTEMIS only works with the files, tabs, and notes that the user explicitly adds to that workspace. It also uses a **preview–approve–undo** approach, meaning that actions that change files or leave the application do not happen automatically; the user sees and approves them first.

For your research area of **voice interaction**, ARTEMIS includes **Speech Input and Output** as Feature 9. This allows the user to **speak a request instead of typing it and hear ARTEMIS's response instead of reading it**. Importantly, your proposal says that speech is not a completely separate capability; it is an alternative way of interacting with the other ARTEMIS features. The same approval process remains in place even when the user interacts through voice.

This means ARTEMIS will cover voice interaction as part of a **larger agentic desktop workflow**. ARTEMIS is not described as simply a voice chatbot. Your proposal says that each request is handled by an agent that reasons about what the user wants, selects the required tools, and can perform a sequence of steps. For example, one request could require ARTEMIS to prepare a summary and then email it to someone.

The agent can break that request into multiple actions, while the approval mechanism keeps those actions bounded and reviewable.

ARTEMIS will also cover several practical tasks around the user's workspace. These include helping the user **resume interrupted work, organize files, combine notes from selected documents, identify repeated manual tasks, understand code and suggest fixes, perform on-demand web searches, draft emails, create document drafts, and interact through speech**. The proposal specifically says that these capabilities are designed to reduce the time and effort users spend on everyday work.

Most importantly for your research, ARTEMIS will evaluate whether its design affects **ease of use and task completion time** for speech-based interaction. Research Objective O9 specifically says that the project will build a speech-based input and output mode and evaluate whether it changes **ease of use or task completion time compared with typed interaction**. The overall evaluation will also consider **task completion time, cognitive workload, and user trust**.

- **Code assistance w/ human review:**
  1-https://arxiv.org/pdf/2411.12924

1. What Already Exists?

The HULA paper shows that **LLM-based software development agents already exist** and are being used to support software engineering tasks. Existing systems such as SWE-agent, AutoCodeRover, RepoUnderstander, Magis, CodeR, and Masai use LLM-based agents to perform tasks such as finding relevant code, generating code, fixing bugs, and solving software development issues. These systems can break a complex software task into different activities and use tools such as code search, compilers, linters, and test suites. In other words, previous research has already established that AI agents can do more than simply answer questions: they can reason about a software task and perform a sequence of actions to help solve it.

The HULA paper itself goes one step further by introducing a **Human-in-the-Loop** approach. HULA has three agents: an **AI Planner Agent**, an **AI Coding Agent**, and a **Human Agent**. The AI Planner identifies relevant files and creates a coding plan, the AI Coding Agent generates code changes, and the human software engineer reviews, modifies, and approves the generated results. HULA therefore does not try to completely replace the human. Instead, it is designed as an assistant in which humans

can provide feedback at different stages. The framework is divided into task setup, planning, coding, and raising a pull request.

The HULA study also shows that this human-in-the-loop approach can work in a real software-development environment. HULA was integrated into Atlassian JIRA and evaluated using both benchmark data and real-world JIRA issues. In the online evaluation, practitioners approved **433 out of 527 generated plans**, giving an **82% plan approval rate**. Of 376 issues for which code was generated, 95 resulted in raised pull requests, and 56 of those pull requests were merged. The practitioners also reported that HULA could reduce development time and effort, particularly for straightforward tasks and for starting a coding plan or writing code.

However, the existing system is mainly focused on **software development**. HULA's workflow is specifically designed around a JIRA issue, source-code repository, file localization, coding plans, code generation, code refinement, and pull requests. It is therefore an example of an AI agent helping a human within a particular professional workflow rather than a general desktop assistant that manages different kinds of everyday work.

---

2. What Is the Research Gap?

The HULA paper identifies several important limitations in previous LLM-based software-development agents. First, earlier systems were mostly evaluated using **historical benchmark datasets and open-source software projects**. This makes it difficult to understand how well these systems work in real enterprise environments. Second, previous work rarely included **human feedback at every stage** of the automated software-development process. Third, many existing systems had **not been deployed in real practice**, so there was limited understanding of how real practitioners perceive their benefits and problems.

HULA addresses these limitations within software development, but the HULA paper itself still identifies problems that remain. One important problem is that the performance of an LLM agent depends heavily on the **amount and quality of input context**. HULA performed better on SWE-bench than on the internal Atlassian dataset. The paper explains that SWE-bench issues generally contain more detailed information, while real JIRA issues can be much shorter. The authors therefore identify a need to understand what information an AI agent needs and how that information can be automatically provided or enhanced.

Another gap is **code quality and complete task completion**. Although HULA can generate plans and code, human involvement is still needed because generated code does not always completely solve the task. In the practitioner survey, only 33% agreed that the generated code solved their task without human involvement, while 67% disagreed that the generated code completely addressed the task. Participants also reported incorrect functionality and incomplete code changes as major challenges. The paper therefore concludes that generating useful code is possible, but generating consistently perfect code without human involvement remains a problem.

The HULA paper also points out a limitation in the way AI systems are evaluated. Passing unit tests alone does not necessarily prove that generated code is completely correct. The authors explain that tests can be expensive to prepare, results are basically pass/fail, and passing tests does not guarantee the absence of defects. They therefore suggest exploring broader ways of evaluating AI-generated work, including aspects such as readability, technical debt, and maintainability.

For ARTEMIS, the important gap comes from this situation: the uploaded HULA paper focuses on AI-assisted **software development**, while your ARTEMIS proposal focuses on a **bounded desktop workspace assistant that supports several types of everyday work**, not only coding. Your proposal specifically describes the problem of people returning to interrupted work without remembering which files they used, what they had decided, or what they planned to do next. It also identifies messy folders, scattered notes, repetitive manual work, routine emails, document creation, web lookup, and difficulty with unfamiliar code as related problems. The proposal states that existing assistants either require an explicit command each time or have broad system access, and that neither approach provides the specific combination of **small trusted memory, workspace boundaries, and everyday task support** that ARTEMIS proposes.

So, **the gap for ARTEMIS is not simply "AI agents do not exist."** They clearly do. The gap identified by your proposal is that the existing approaches described there do not provide the particular combination of a **bounded, opt-in workspace + visible knowledge + user approval + undo + session resumption + multiple everyday work tasks** in one desktop assistant. ARTEMIS is designed around this combination.

---

3. What Will ARTEMIS Cover?

ARTEMIS will cover a broader set of tasks within a **bounded and user-controlled workspace**. The user first selects a folder and adds it as a workspace. ARTEMIS only

works with files, tabs, and notes that the user explicitly adds to that workspace. It does not read outside the selected folder. The proposal also states that actions that change files or leave the application, such as sending an email, require the user to see and approve the action first. The system also provides an **“What ARTEMIS Knows”** panel so that the user can see what information the system currently holds and remove individual items.

The first major feature is **“Pick Up Where You Left Off.”** This feature is specifically designed for the problem of interrupted work. ARTEMIS will show recently changed files, saved notes, and what the user said they were going to do next, allowing the user to reopen files quickly. Importantly, your proposal states that this feature does **not use AI interpretation**. Its purpose is simply to restore the user's previous work context quickly instead of making the user manually search through files again.

ARTEMIS will also cover **file organization and document-based work**. Its “Tidy Up a Folder” feature reads file names and short text extracts and proposes a grouping before anything is moved. The user must approve the proposed organization, and an undo option can restore the previous state. Another feature, “Pull Together Notes From a Few Documents,” combines key points or citations from a small, explicitly selected group of documents into a new file without overwriting existing files.

For repetitive work, ARTEMIS will include **“Notice a Repeated Task.”** When the same manual pattern occurs three or more times, ARTEMIS asks the user whether they want to automate it. It does not automatically decide to automate the task. The proposal specifically says that the user must approve the automation and that automatic runs can be undone. This continues the project's main principle of keeping the user in control.

ARTEMIS will also include a **bounded Code Assistant**. It can read source files that are already inside the selected workspace and answer questions or explain errors. If it suggests a code fix, the proposed change is displayed as a diff and is applied only after the user approves it. Unlike a fully autonomous coding agent, the proposal explicitly states that ARTEMIS will not write or run code on its own.

In addition, ARTEMIS will provide **on-demand web search, mailing, document drafting, and speech input/output**. Web search is only performed when the user explicitly asks for information that is not available in the workspace. Mailing produces a complete draft for the user to review, and the message is sent only after approval. Document drafting creates an editable first draft from workspace material rather than automatically finalizing or sending anything. Speech input and output allows the user to speak requests and hear responses while following the same approval process as typed interaction.

Finally, ARTEMIS will use **agentic AI reasoning** to connect these capabilities. According to your proposal, ARTEMIS is not intended to be only a question-answer chatbot. The agent reasons about what a request requires, selects available tools, and decides whether several steps are needed. For example, a request to prepare a summary and email it can involve document information, synthesis, drafting, and mailing as a sequence of actions. However, ARTEMIS remains bounded because actions that change files or leave the workspace require user approval before execution.

2-https://arxiv.org/pdf/2505.22906

1. What Already Exists?

The HILDE paper explains that AI programming assistants already exist and are widely used to help programmers generate code. Tools such as GitHub Copilot can generate code completions and can also provide several alternative completions. However, the paper explains that these systems mainly focus on **speed and productivity**, while users can become less involved in important programming decisions. The AI may make decisions about security, efficiency, readability, coding style, and other requirements without clearly showing the user what other choices were available. Because of this, programmers may simply accept the AI-generated solution without critically comparing it with other possibilities.

Previous research discussed in HILDE has already explored several ways of improving AI programming assistants. Some research focuses on **making AI-generated code more secure**, while other work uses LLMs to detect or repair vulnerabilities. There is also research on **steering LLMs**, where users can interact with the AI during the programming process and try to clarify their intentions. Other systems provide multiple AI-generated alternatives or visualizations so that users can understand that an LLM can produce different solutions. There has also been research on highlighting the uncertainty of LLM-generated code.

HILDE itself already improves on these approaches by introducing **Human-in-the-Loop Decoding**. It highlights important decision points in generated code, shows local alternatives that the LLM considered, and gives explanations of how those alternatives differ. The user can then select a preferred alternative at the token level, and HILDE regenerates the following code according to that choice. In the study, this helped programmers become more involved in the AI's decisions instead of simply accepting the first suggestion.

The HILDE study showed that this approach can improve security-related programming. In the study with 18 participants, users generated **31% fewer vulnerabilities on average** with HILDE than with the baseline assistant. Users also intentionally repaired more vulnerabilities using HILDE. The paper therefore demonstrates that showing alternatives, explaining their differences, and allowing users to make fine-grained decisions can make interaction with an AI programming assistant more intentional.

---

2. What Is the Gap?

The important gap identified from the HILDE paper is that existing AI programming assistants mainly concentrate on **code generation and programming**, while users still have limited control over the AI's individual decisions. Traditional assistants provide global alternatives, but these alternatives may be similar to one another and users may have difficulty understanding the meaningful differences between them. The HILDE paper specifically explains that users often have to reverse-engineer the AI's choices because the alternatives do not clearly show what important changes they contain.

Another gap is that simply showing uncertainty is not enough. Earlier work found that uncertainty highlighting alone did not provide an advantage over a baseline. HILDE addresses this by combining uncertainty with the **semantic importance of alternatives** and providing natural-language explanations. This means that HILDE itself fills a gap in AI coding interfaces by helping users understand important decisions rather than simply telling them that the model is uncertain.

However, the HILDE paper is still mainly focused on **code generation within the programming environment**. Its current implementation is a Visual Studio Code extension designed around programming tasks, particularly security-related tasks. The authors themselves state that the approach could be adapted to other priorities such as efficiency, maintainability, energy conservation, and personal coding style, but making it more general and customizable is future work.

The paper also identifies practical limitations. HILDE can sometimes make unwanted changes to downstream code when a user selects a local alternative. The regeneration process can overwrite previous user edits, and the authors identify preserving incremental user modifications as an open challenge. In addition, some participants found the number of alternatives and token highlighting overwhelming at times.

For your **ARTEMIS project**, the larger gap is therefore broader than HILDE's programming-specific problem. Your proposal describes a problem where people lose time after interruptions because they cannot easily remember which files they were using, what they had already decided, and what they intended to do next. It also identifies problems such as messy folders, scattered notes, repetitive manual work, difficulty with unfamiliar code, needing outside information, writing routine emails, creating documents, and inconvenient typing/reading. Your proposal states that existing assistants either require an explicit command each time or operate with broad access to the user's system, and neither approach provides the specific combination of **work-resumption, bounded access, everyday task support, and user control** that ARTEMIS aims to provide.

---

3. What Will ARTEMIS Cover?

ARTEMIS will cover this gap by going beyond a programming-only AI assistant and providing a **bounded, opt-in desktop assistant** for managing work across a selected workspace. According to your proposal, the user explicitly selects a folder and adds it as a workspace. ARTEMIS only knows about files, tabs, and notes that the user has explicitly added, and it does not read anything outside the chosen folder. It also uses a **preview–approve–undo** model: actions that change files or leave the application are shown to the user first and require approval. The proposal also includes a “What ARTEMIS Knows” panel so the user can see what information the system currently holds and delete individual items.

The first major area ARTEMIS will cover is **resuming interrupted work**. Its “Pick Up Where You Left Off” feature will show recently changed files, saved notes, and what the user said they were about to do next. The user can reopen files with one click. Importantly, your proposal says that this feature does **not use AI interpretation**; it is intended simply to restore the user's working context quickly. This directly addresses the problem of losing time reconstructing work after a break.

ARTEMIS will also cover **file organization and repetitive work**. Its “Tidy Up a Folder” feature will read file names and short document extracts and propose a grouping before anything is moved. The user must approve the proposed organization, and an undo can restore the previous state. Similarly, “Notice a Repeated Task” will identify a repeated manual pattern after it happens three or more times and ask whether the user wants to automate it. ARTEMIS will not automatically impose the automation, and automatic runs can be undone.

Another area ARTEMIS will cover is **working with multiple documents and information**. It will allow users to select a small set of documents and combine their citations or key points into a new file. Existing files will not be overwritten. ARTEMIS will also provide on-demand web search when the information cannot be found inside the workspace. The important point in your proposal is that this is **direct and user-requested**, rather than continuous background browsing.

ARTEMIS will also include a **bounded code assistant**, but its purpose is different from HILDE. ARTEMIS will read source files already inside the selected workspace, answer questions, and explain errors. If it suggests a fix, the fix will be presented as a **diff**, and the user must approve it before it is applied. ARTEMIS will not independently write or execute code. Therefore, ARTEMIS takes the idea of keeping the user involved in AI-assisted programming and places it inside a broader workspace-based assistant with explicit boundaries and approval.

Finally, ARTEMIS will cover everyday tasks that are outside the programming focus of HILDE. It will provide **email drafting, document drafting, web search, and speech input/output**. For email, ARTEMIS prepares a draft from information already available in the workspace, shows the complete draft to the user, and sends it only after approval. For documents, it creates an editable first draft based on workspace material. Speech allows users to speak requests and hear responses while following the same approval rules.

3-https://arxiv.org/pdf/2505.20206

1. What Already Exists?

The research paper shows that **AI and Large Language Models (LLMs) are already being used to support and automate code review**. Earlier approaches focused on tasks such as recommending suitable human reviewers, generating review comments, predicting whether a code change should be approved, and automatically modifying source code. The paper mentions systems such as CommentFinder, AUGER, T5-based code review automation, AutoTransform, and LLM-based code review agents. It also explains that tools powered by LLMs are being used to reduce the manual effort required in code review.

More specifically, the paper evaluates **GPT-4o and Gemini 2.0 Flash** for code review. The models are asked to determine whether code is correct or incorrect and, when necessary, provide a corrected version. The study found that these LLMs can perform code correctness assessment and can also generate code improvements. For example, on the mixed dataset, GPT-4o achieved **68.50% correctness accuracy** when a

problem description was provided, while Gemini achieved **63.89%**. GPT-4o also corrected **67.83%** of incorrect code, compared with **54.26%** for Gemini.

The paper also shows that **context is important for LLM-based code review**. When problem descriptions were provided, the models generally performed better. When the descriptions were removed, their performance decreased. The paper therefore identifies code comments and pull-request descriptions as useful information for helping LLMs understand the intended functionality of the code.

Your A.R.T.E.M.I.S proposal already includes a **Code Assistant** that can read source files inside a selected workspace, answer questions about the code, explain errors, and suggest fixes. The proposed fix is shown as a diff and is only applied after the user approves it. Therefore, A.R.T.E.M.I.S already uses the idea of an AI assistant helping users understand and improve code, but it places this capability inside a **bounded workspace and user-approval framework**.

---

2. What Is the Gap?

The main gap identified by the research paper is that **LLM-based code review is not reliable enough to be completely automated**. Although GPT-4o and Gemini can identify incorrect code and suggest corrections, they still make significant mistakes. The highest correctness accuracy reported in the study was only **68.50%**, and the models could also make incorrect suggestions for code that was already correct. The paper reports regression rates reaching up to **24.80%** in its discussion of code suggestions. Because of these errors, the authors state that fully automated LLM code review may be unreliable.

Another important gap is **human control and oversight**. The paper explains that traditional code review is not only about finding technical errors. It also supports knowledge transfer, team awareness, and shared code ownership. A fully automated system may reduce manual work, but it can lose these human-driven benefits. Because of this, the paper proposes a **Human-in-the-loop LLM Code Review** process in which the LLM performs the review, but a human remains responsible for deciding whether further human review is needed.

The paper also identifies a **generalization gap**. The experiment was limited to Python and tested only GPT-4o and Gemini. The datasets included HumanEval solutions and AI-generated code, while the authors specifically state that future experiments should use code from **real software projects** for deeper and more reliable insights. They also explain that different prompts and different datasets can produce different results.

For your A.R.T.E.M.I.S literature review, this creates a clear connection: **existing LLM code-review research focuses mainly on evaluating whether LLMs can review and correct code, while A.R.T.E.M.I.S focuses on how an AI assistant can provide this type of help within a controlled workspace and with the user remaining in control.** The A.R.T.E.M.I.S proposal specifically identifies distrust of AI systems with broad system access as a problem and proposes a bounded workspace, preview/approval, and undo mechanisms to address it.

---

3. What Will A.R.T.E.M.I.S Cover?

A.R.T.E.M.I.S will cover the **code-assistance part** of this gap through its Code Assistant feature. It will read source files that are already inside the user's selected workspace and answer direct questions about the code, including explaining errors. When a fix is needed, A.R.T.E.M.I.S will show the suggested change as a **diff**, and the change will only be applied after the user approves it. The system will **not write or run code on its own**.

More importantly, A.R.T.E.M.I.S will cover the **human-control problem** identified in the research paper. The whole system is designed as a bounded, opt-in desktop assistant. The user explicitly selects a folder as a workspace, and A.R.T.E.M.I.S only works with files, tabs, and notes that have been explicitly added to that workspace. It does not read outside the selected folder. Actions that change files or leave the application require the user to see and approve them first, and an undo mechanism is provided afterward.

A.R.T.E.M.I.S will also extend this controlled approach beyond code review. Its planned features include **resuming interrupted work, organizing folders, combining information from selected documents, detecting repeated tasks, web search, email drafting, document drafting, and speech input/output**. These features are connected through the same approval-based approach rather than giving the AI unrestricted control over the user's computer.

The research paper proposes human involvement mainly because LLM code-review decisions and suggestions can be wrong. A.R.T.E.M.I.S takes a similar principle and makes **approval and user control a central part of the whole assistant**. Its research objectives specifically include building a bounded workspace model, evaluating task resumption, creating preview-approve-undo interactions, building a bounded code assistant, and evaluating the system in terms of **task completion time, cognitive workload, and user trust**

4-https://ojs.aaai.org/index.php/AAAI/article/view/28878

1. What Already Exists?

The paper **"When to Show a Suggestion? Integrating Human Feedback in AI-Assisted Programming"** presents an AI-assisted programming system, specifically GitHub Copilot, that gives code suggestions to programmers while they are writing code. The system normally shows a suggestion when the programmer pauses typing. The programmer can then either accept the suggestion or reject it by continuing to type. The paper uses this programmer behavior as **human feedback**. In other words, the system can learn from whether programmers accept or reject the suggestions. The researchers collected telemetry data from real programming sessions, including the programmer's prompts, suggestions, and actions. Their dataset contained data from **535 programmers**, with more than **168,000 shown suggestions**.

The main existing contribution in this paper is **Conditional Suggestion Display from Human Feedback (CDHF)**. Instead of showing every generated suggestion, CDHF tries to decide whether a suggestion should be shown or hidden. It uses machine-learning models to predict the probability that a programmer will accept a suggestion. If the system predicts that the programmer is likely to reject it, the suggestion can be hidden. The approach also has a two-stage design. First, it can make a decision using only the code and previous interaction information, without generating the suggestion. If a decision cannot be made confidently, the system generates the suggestion and then uses a second model that also considers the actual suggestion. This can reduce unnecessary suggestion generation and therefore reduce latency.

The paper also studies the **programmer's latent state**. The authors explain that telemetry does not show what the programmer is actually doing between two recorded events. For example, the programmer may be checking documentation, thinking about the code, or verifying a suggestion. Their earlier study showed that including this latent state can improve prediction of whether a programmer will accept a recommendation. In their comparison, the model without latent state achieved **61.9% accuracy**, while the model with latent state achieved **83.6% accuracy**. This shows that programmer behavior and context can be important when deciding how an AI assistant should interact with a person.

The paper also discusses an important limitation of using acceptance as the main reward. If an AI system is trained mainly to maximize acceptance, it may prefer **shorter suggestions**, because shorter pieces of code may be easier for programmers to accept. Their experiment found evidence that optimizing only for acceptance can favor very short or even single-token suggestions instead of complete useful code. The

authors therefore identify a potential problem with simply treating user acceptance as the definition of a good AI suggestion.

So, in simple words, **what already exists is an AI coding assistant that learns from human accept/reject behavior and uses that information to decide when a code suggestion should be displayed.** It focuses mainly on improving the interaction between the programmer and the AI coding assistant by reducing unnecessary suggestions, reducing verification effort, and reducing latency.

---

2. What Is the Gap?

The main gap is that this paper focuses on **one specific AI-assisted programming interaction: deciding when to show or hide code suggestions**. The CDHF method is designed around Copilot's suggestions and programmer acceptance/rejection behavior. It does not provide a broader desktop assistant that can manage different types of work across files, notes, documents, web search, email, and other everyday tasks. The paper itself says that its focus is on **when to display suggestions**, and that it does **not tackle which suggestions to display among a candidate set**.

Another gap is that the system mainly uses **telemetry data** to understand programmer behavior. The authors explicitly explain that telemetry cannot capture the programmer's activities and thinking between two recorded events. They call this the programmer's **latent state**. Although the paper shows that latent state can improve acceptance prediction, the actual CDHF system is built using the telemetry information available to it. This leaves a gap in understanding and supporting the user's broader working context.

There is also a gap regarding **real-world evaluation**. The CDHF results are mainly based on a **retrospective evaluation**, meaning the researchers used previously collected interaction data to estimate what would have happened if CDHF had been used. The authors themselves state that a user study is required to verify whether the method actually makes programmers more productive. They also warn that hiding useful suggestions or relying too heavily on acceptance rate can make the experience worse.

Most importantly for your ARTEMIS literature review, this paper does not address the problem of **resuming interrupted work across a user's workspace**. It does not provide a mechanism that remembers which files a person was working on, what notes

were saved, or what the person planned to do next. It also does not address organizing a folder, collecting information from several documents, detecting repeated manual tasks, drafting emails, drafting documents, or providing speech interaction. These are outside the specific problem addressed by this paper.

Therefore, the gap that is relevant to ARTEMIS is that **existing work demonstrates how human feedback can improve one particular AI interaction, but it does not provide a bounded, workspace-based assistant that uses context and controlled agentic actions to support the user's complete workflow.** ARTEMIS is designed to address this broader workspace-level problem.

---

3. What Will ARTEMIS Cover?

ARTEMIS will cover a **broader and more practical workspace-level form of AI assistance**. According to your ARTEMIS proposal, ARTEMIS is a **bounded, opt-in desktop assistant**. Instead of having access to the whole computer, ARTEMIS only works with files, tabs, and notes that the user explicitly adds to a workspace. It also provides a **“What ARTEMIS Knows”** panel so the user can see what information the system currently has. The user can delete individual items from this knowledge.

A major area ARTEMIS will cover is **resuming interrupted work**. The “Pick Up Where You Left Off” feature shows recently changed files, saved notes, and what the user said they were going to do next. The purpose is to help the user reconstruct their previous working context quickly instead of manually searching through files and trying to remember what they were doing. Importantly, this feature does not depend on AI interpretation; it uses the recorded workspace information to restore the user's context.

ARTEMIS will also cover **file organization and document-based work**. It can propose groups for messy files based on file names and short text extracts, but it does not move anything automatically. The user first sees the proposed organization and approves it. ARTEMIS can also combine notes, citations, or key points from a small set of selected documents into a new file without overwriting the original files.

Another important part of ARTEMIS is **controlled automation**. If ARTEMIS notices that the same manual task has happened three or more times, it can ask the user whether they want to automate it. It does not automatically decide to automate the task. Similarly, actions that change files or leave the application require user approval and

can be undone. This gives ARTEMIS a different interaction model from a system that acts without the user's confirmation.

ARTEMIS will also include a **bounded code assistant**. It can read source files that are already inside the selected workspace and answer questions about the code or explain errors. If it proposes a fix, the fix is shown as a **diff**, and the user must approve it before it is applied. ARTEMIS therefore extends the idea of AI-assisted programming from simply deciding whether to show a suggestion toward a workspace-based assistant where code assistance is one part of a larger system.

In addition, ARTEMIS will cover **on-demand web search, email drafting, document drafting, and speech input/output**. Web search is only performed when the user explicitly asks for information that is not available in the workspace. Email is drafted from workspace material and shown to the user before sending. Documents are created as editable drafts rather than being finalized automatically. Speech allows users to speak requests and hear responses while following the same approval process.

The most important contribution of ARTEMIS in relation to this paper is therefore its **bounded and approval-based agentic approach**. ARTEMIS is not only predicting whether an AI response will be accepted. It has an agent that can reason about a request, select available tools, and perform multiple steps, but actions that change files or leave the workspace stop for user approval. The proposal specifically describes this as different from both a normal chatbot and a fully autonomous agent: ARTEMIS can plan multiple actions, but its autonomy remains bounded and reviewable.

5-https://aclanthology.org/2024.findings-emnlp.908.pdf

1. What Already Exists?

The paper shows that existing Large Language Models (LLMs) such as **GPT-4, Gemini Ultra, and Claude 3.5 Sonnet** can generate code from natural language, but they have difficulty solving complex programming problems, especially medium and hard competition-level problems. Existing approaches already try to improve code generation through techniques such as **self-debugging, self-reflection, Chain-of-Thought (CoT), Tree-of-Thought, and human feedback**. In self-debugging and reflection approaches, the model examines its own generated code, identifies possible problems, and tries to improve the code. CoT and Tree-of-Thought help models break difficult problems into smaller reasoning steps. Previous research also showed that humans can provide feedback to LLMs to improve or correct generated code. For example, human feedback can clarify an unclear requirement or correct a small error in the generated code.

The paper specifically studies **human steering of code-generating LLMs**. Eight expert programmers interacted with GPT-4, Gemini Ultra, and Claude 3.5 Sonnet on difficult LeetCode problems. The programmers used multiple conversational turns to guide the models when their initial solutions failed. The study identified different strategies, including **test reiteration, new test definition, revising unit tests, pointing out a specific programming error, addressing code inefficiency, requirement reiteration, requirement clarification, approach re-orientation, and specific approach instruction**.

So, in simple words, **what already exists is AI code generation plus different methods for improving the generated code, including self-reflection, prompting, and human feedback.** The paper also shows that humans can successfully guide an LLM through several rounds of feedback instead of expecting the model to solve a difficult problem correctly in one attempt.

---

2. What Is the Gap?

The main gap identified by this paper is that, although previous research has studied **code generation, self-debugging, reflection, and human feedback**, there was limited understanding of **what specific feedback strategies expert users actually use to steer an LLM toward a successful solution**. The authors explicitly state that, to their knowledge, previous work had not explored the different types of feedback provided by expert users to guide these models more effectively toward a solution. Therefore, this paper fills that gap by observing real conversations between expert programmers and code-generating LLMs and categorizing the steering strategies they use.

Another gap is that existing work had demonstrated human-model collaboration, but this paper focuses specifically on **highly difficult, competition-level programming problems** rather than simpler coding tasks. The authors explain that their focus differs from earlier work because they study competition-level problems, which are more complex than the problems in datasets such as MBPP.

The study also identifies an important limitation: it is **not yet clear whether novice programmers would achieve the same level of success as expert programmers**. Some strategies, especially **“Point out specific error”** and **“Approach Re-orientation,”** may require expert programming knowledge. The authors therefore suggest future research with novice users. They also mention that their dataset is relatively small and that more data is needed to make broader generalizations.

Most importantly for your literature review, the paper's future work suggests that the identified strategies could eventually be used to design **Human-LLM interfaces that recommend follow-up prompts, ask clarifying questions, or provide prompt templates** based on the Socratic feedback approach. The authors also suggest dynamically adapting steering strategies according to the model's response and the changing conversation context.

In simple words, the gap is: existing systems can generate and improve code, and humans can give feedback, but there is not yet enough understanding or system-level support for **automatically helping users decide what kind of feedback or next action should be given when an LLM gets stuck.** This paper mainly *studies and categorizes* those human steering strategies rather than building them into a complete bounded desktop assistant.

---

3. What Will A.R.T.E.M.I.S Cover?

A.R.T.E.M.I.S will cover a **broader agentic workspace-assistance problem** rather than focusing only on code generation. According to your proposal, A.R.T.E.M.I.S is a bounded desktop assistant where the user explicitly selects a workspace, and the system only works with files, tabs, and notes that the user has added to that workspace. It is designed to help users resume interrupted work, organize files, combine information from documents, detect repetitive tasks, understand code, search the web when explicitly asked, draft emails, create documents, and interact through speech.

For the **code-assistance part**, A.R.T.E.M.I.S will read source files that are already inside the selected workspace and answer direct questions, including explaining errors. It can suggest a fix as a **diff**, but the fix is applied only after the user approves it. It does not independently write or execute code. This means that the work in the SoHF paper is relevant to A.R.T.E.M.I.S because it demonstrates that **iterative human guidance can help an LLM overcome coding failures**. A.R.T.E.M.I.S can use this general idea within its bounded code-assistant feature, while keeping the user in control of proposed changes.

However, A.R.T.E.M.I.S is not proposed as a system that specifically implements all of the SoHF strategies. Your proposal says that A.R.T.E.M.I.S will be an **agentic AI system** that reasons about a user's request, chooses available tools, and can perform multiple steps, but actions that modify files or leave the workspace require user approval. Therefore, based strictly on your proposal, the key thing A.R.T.E.M.I.S will cover is **bounded, reviewable, user-controlled AI assistance across a workspace**,

including code assistance, rather than reproducing the exact SoHF experimental framework.

6-https://link.springer.com/article/10.1007/s10515-026-00638-5?utm_source=chatgpt.com

1. What Already Exists?

Existing research already shows that **LLMs can be used for code review, code understanding, code generation, and checking whether code follows a given requirement**. Earlier systems and models such as CodeBERT, GraphCodeBERT, CodeT5, Codex, AlphaCode, CodeGen, InCoder, CodeLlama, and systems such as SWE-bench have demonstrated that language models can understand code, generate code, explain code, suggest changes, and help with debugging and software development tasks. The paper also explains that recent research has started using LLMs as **"virtual reviewers"**, where an LLM receives a natural-language requirement and code and decides whether the code satisfies that requirement. Some work has also investigated LLMs for verifying requirements even when complete test cases or formal implementations are not available.

Another existing direction is **LLM-as-a-judge**. In this approach, an LLM is used to evaluate or judge another output instead of relying only on traditional evaluation methods. Researchers have used prompting methods such as explanation-based prompting, chain-of-thought-style reasoning, self-consistency, Self-Refine, Reflection, and ReAct to make LLMs reason more carefully. The general idea behind these approaches is that if an LLM is asked to explain its decision, identify problems, or propose a fix, it may produce a better and more reliable answer.

The paper, however, shows that these existing approaches are **not completely reliable** for requirement-conformance judgement. LLMs can sometimes reject correct code or accept buggy code. In particular, the experiments found that more detailed prompts can actually increase the tendency to reject correct implementations. For example, GPT-4o's false-negative rate increased strongly when moving from a simple judgement prompt to prompts requiring explanations and fixes. This means that existing LLM-based code-review approaches can provide useful code analysis, but their decisions and explanations cannot automatically be considered trustworthy.

From the ARTEMIS proposal, existing AI assistants also provide different types of assistance, including assistants and computer-use agents from Microsoft, Google, OpenAI, and Anthropic. According to the proposal, these existing assistants generally

either require the user to give an explicit command each time or can operate with broad access to the user's system. The proposal identifies that these approaches do not specifically solve the problem of **resuming interrupted work with a small, trustworthy memory while also helping with everyday tasks such as file organization, searching, coding, document drafting, and communication**.

---

2. What Is the Gap?

The main gap identified by the research paper is that **LLMs are not reliably able to judge whether code follows a natural-language requirement**, especially when there are no test cases or formal specifications available for the judgement. Existing research has shown that LLMs have strong code-generation and code-understanding abilities, but much less is known about whether their **judgements themselves are reliable**. The paper specifically investigates this problem and finds that LLMs can make two important types of mistakes: **false rejection**, where correct code is judged as incorrect, and **false acceptance**, where buggy code is judged as correct.

A particularly important gap is **over-correction bias**. The paper finds that LLMs often become too strict. Instead of simply checking whether the code satisfies the requirement, they may invent requirements, worry about unspecified edge cases, assume different input types, imagine runtime problems, or claim that the algorithm is wrong even when the implementation is actually correct. The study found that the largest category of these false-rejection explanations was **Logic Error**, followed by **Added Requirement, Boundary Error, and Misread Specification**. Together, these four categories represented 87.2% of the false-negative explanations in the study.

Another gap is that **asking an LLM for more reasoning does not necessarily make its reasoning more trustworthy**. The research compared three prompting approaches: Direct, Direct+Explain, and Full. The Full prompt asks the model to judge the code, explain its decision, and propose a fix. The results show that increasing prompt complexity can reduce false acceptance in some situations but can also greatly increase false rejection. In other words, more detailed reasoning can move the model toward excessive conservatism rather than simply improving accuracy.

There is also a gap in the **reliability of explanations**. The paper evaluates whether the explanation actually supports the model's YES/NO decision. It finds that explanations can contradict the model's own final judgement. For example, GPT-4o often produced cases where it answered **NO**, while its own explanation was actually positive toward the

code. Other models showed the opposite pattern: they answered **YES** while their explanations still described possible problems. Therefore, a longer explanation should not automatically be treated as evidence that the decision is trustworthy.

The paper also identifies a gap between recognizing **what went wrong** and understanding **why it went wrong**. The models were generally much better at identifying the observable symptom of a bug than identifying the actual underlying bug type. For example, GPT-4o had very high symptom-match rates, but its bug-type match was considerably lower. This means an LLM may correctly say that a program produces an incorrect output but incorrectly explain the underlying cause.

For ARTEMIS specifically, the proposal identifies a broader practical gap: users may lose time when they return to interrupted work, have messy folders, have notes scattered across documents, repeat manual tasks, get stuck with unfamiliar code, need outside information, or have to write routine emails and documents. Existing assistants, according to the proposal, do not combine these needs with a **small, explicitly shared workspace and strong user control**. The proposal therefore identifies a gap between broad AI assistance and a bounded assistant that remembers only what the user intentionally shares and requires approval before actions are taken.

---

3. What Will ARTEMIS Cover?

ARTEMIS will address this gap by providing a bounded, opt-in desktop assistant rather than a system with unrestricted access. The user first selects a folder and makes it a workspace. ARTEMIS only works with files, tabs, and notes that have explicitly been added to that workspace. It does not read outside the selected folder. The system also provides a **“What ARTEMIS Knows”** panel so the user can see what information the system currently has and can delete individual items. This directly addresses the proposal's focus on keeping the assistant's knowledge small, visible, and controlled.

ARTEMIS will also cover **resuming interrupted work** through its “Pick Up Where You Left Off” feature. It will show recently changed files, saved notes, and what the user said they intended to do next, allowing the user to reopen relevant files quickly. Importantly, the proposal states that this feature does not use AI interpretation. ARTEMIS will also help organize messy folders by reading file names and short text extracts and then proposing a grouping as a preview. The files will not be moved until the user approves the action, and an undo will be available.

ARTEMIS will further cover **document and information management**. It can combine citations or important points from a small set of explicitly selected documents into a new file, without overwriting existing files. It can also detect repeated manual tasks when the same pattern happens three or more times and ask the user whether they want to automate it. The automation is not imposed automatically; the user must approve it, and automatic runs can be undone.

For **code assistance**, ARTEMIS will read source files that are already inside the workspace and answer direct questions, including explaining errors. It can suggest a fix, but the fix is shown as a **diff** and is applied only after user approval. The proposal specifically says that ARTEMIS does not write or run code on its own. This is particularly relevant to the code-review paper because that paper shows that LLM-generated fixes and judgements can be unreliable. ARTEMIS's design therefore keeps the suggested code change reviewable and user-controlled rather than automatically applying it.

ARTEMIS will also cover **on-demand web search, mailing, document drafting, and speech interaction**. Web search happens only when the user explicitly asks for information that is not available in the workspace. Mailing creates a draft from information already available in the workspace, shows the complete draft to the user, and sends it only after approval. Document drafting creates a new editable first draft rather than automatically finalizing anything. Speech input and output provide another way to interact with the same features while keeping the same approval requirements.

Finally, the major idea connecting all these features is **bounded agentic AI with user approval**. ARTEMIS is not designed as only a question-answer chatbot. According to the proposal, an agent receives the user's request, reasons about what is needed, chooses the appropriate tools, and can perform multiple steps. However, it is also not intended to be a fully autonomous agent. Any action that changes a file or leaves the workspace stops for user approval before execution. The proposal gives the example of preparing a summary and emailing it to a supervisor: ARTEMIS can plan the required sequence of actions, but the user remains in control of the actions that actually happen.

- **Hallucinations in ai summarization** for feat 3
- **Rag citation accuracy**

Theme: Hallucinations in AI summarization

Quick definition check before the papers, since it matters here: factuality means whether a statement is true in the real world. Faithfulness means whether a statement is actually supported by the source document, whether or not it happens to also be true. A

summary can be faithful but factually wrong if the source itself is wrong, and it can be factually true but unfaithful if the model added something correct that just wasn't in the document. Every paper below is really studying faithfulness, not factuality, even when they use the word "factual."

Maynez, Narayan, Bohnet, and McDonald, "On Faithfulness and Factuality in Abstractive Summarization," ACL 2020.
https://aclanthology.org/2020.acl-main.173/ · DOI: 10.18653/v1/2020.acl-main.173

What the paper does. They ran a large human evaluation on several neural abstractive summarization systems, having annotators check whether each generated summary actually matched its source document. They split hallucinations into two types, intrinsic, where the model twists or misstates something that was actually in the source, and extrinsic, where the model adds something that wasn't in the source at all. They found substantial amounts of hallucinated content in every single system they tested. They also checked whether the standard metric, ROUGE, could catch this problem, and it mostly couldn't. What did correlate better with human judgments of faithfulness was textual entailment, basically checking whether the source document logically implies the summary sentence.

The gap. This paper only measures and describes the problem. It doesn't build a system that produces faithful summaries, and it doesn't build an automatic detector either, it just shows that entailment is a promising direction for one. So after this paper, you know hallucination is common and you know ROUGE won't catch it, but you still don't have a tool that catches it for you.

ARTEMIS. Feature 3 in the proposal pulls together notes and citations from a small, explicitly selected set of documents, which is a narrower, more controlled version of the abstractive summarization task this paper studies on open datasets like XSum. Narrower scope reduces the surface area for hallucination somewhat, but the proposal doesn't describe anything like an entailment check or any other mechanism to catch intrinsic or extrinsic hallucination if it happens. This paper's finding, that hallucination shows up in every system tested, is a real reason to expect Feature 3 needs some kind of check like this, not a solved problem ARTEMIS already handles.

Kryscinski, McCann, Xiong, and Socher, "Evaluating the Factual Consistency of Abstractive Text Summarization," EMNLP 2020.
https://aclanthology.org/2020.emnlp-main.750/ · DOI: 10.18653/v1/2020.emnlp-main.750

What the paper does. This one actually builds something, a model called FactCC that predicts whether a summary sentence is factually consistent with its source. Since there

wasn't much real training data for this, they generated their own by applying rule-based transformations to source document sentences, swapping entities, negating statements, and so on, to create realistic-looking inconsistent examples. The model is trained to do three things at once: say whether a sentence is consistent, point to the part of the source that supports or contradicts it, and if it's inconsistent, point to exactly what part of the summary is wrong.

The gap. FactCC is a detector, not a fix. It tells you after the fact that something is wrong, it doesn't stop the wrong summary from being generated in the first place. There's also a real question about whether training on synthetic rule-based errors, entity swaps, negations, teaches the model to catch the kinds of subtler mistakes real generation models actually make, which don't always look like a simple rule-based edit.

ARTEMIS. This is the closest thing in the literature to a plug-in tool ARTEMIS's Feature 3 could actually use, a lightweight, already-built consistency checker rather than something that would need to be built from scratch. Nothing in the proposal currently describes using something like FactCC, or any detector at all, so this stays a genuine open gap in the current design rather than something Feature 3 already does. If the team wanted to reduce hallucination risk in Feature 3, adopting a FactCC-style check after generation is a concrete, literature-backed way to do it, not something ARTEMIS has already built in.

Pagnoni, Balachandran, and Tsvetkov, "Understanding Factuality in Abstractive Summarization with FRANK: A Benchmark for Factuality Metrics," NAACL 2021. https://aclanthology.org/2021.naacl-main.383/ · DOI: 10.18653/v1/2021.naacl-main.383

What the paper does. Earlier work, including the two papers above, mostly treated factuality as a yes-or-no question. This paper argues that's too coarse, and builds a typology of specific error types instead, things like wrong entities, wrong relations between entities, wrong dates, and so on. They collect human annotations of these specific error types on real model-generated summaries from CNN/DM and XSum, then use that annotated data as a benchmark to test how well existing automatic factuality metrics, including FactCC, actually agree with human judgment.

The gap. The headline finding is that no single metric is reliably good across all error types, different metrics catch different kinds of mistakes and miss others. This is a meta-evaluation paper, it evaluates the evaluators, it doesn't propose a new generation or detection method itself. So what this adds to the picture is that even the detection side of this problem, which sounds more solved than the generation side, is actually fragmented, you can't just pick one off-the-shelf metric and trust it uniformly.

ARTEMIS. This paper's contribution is mainly a warning for anyone trying to solve Feature 3's hallucination risk with a single automatic metric. If the team ever does add a consistency check to Feature 3, this paper is a reason not to assume one metric, FactCC or anything else, will catch every kind of error a citation-synthesis feature could produce. It doesn't hand ARTEMIS a gap to fill so much as it tells you to expect any single fix to be incomplete.

Laban, Kryscinski, Agarwal, Fabbri, Xiong, Joty, and Wu, "SummEdits: Measuring LLM Ability at Factual Reasoning Through The Lens of Summarization," EMNLP 2023. https://aclanthology.org/2023.emnlp-main.600/ · DOI: 10.18653/v1/2023.emnlp-main.600

What the paper does. This is the most recent and most relevant one, since it tests actual LLMs, the kind ARTEMIS would be built on, rather than older summarization-specific models. They found existing benchmarks for testing inconsistency detection were flawed in ways that skewed results, so they built a new one, SummEdits, across ten domains, designed to be cheap to build and highly reproducible, with annotator agreement around 0.9. Then they tested a range of LLMs on it, checking whether the models could correctly tell a consistent summary from an inconsistent one.

What they found. Most LLMs performed close to random chance at this task. Even GPT-4, the best model they tested, still landed 8% below estimated human performance.

The gap. This is the clearest, most current evidence that catching hallucination automatically is still unsolved, even by the most capable general-purpose models available at the time of the study, not just by older specialized detectors like FactCC. If GPT-4 struggles to reliably tell a faithful summary from an unfaithful one, that's a direct warning for any system, including ARTEMIS, that plans to use an LLM to check its own citation-grounded output.

ARTEMIS. This paper is the strongest reason to treat O4's evaluation plan, measuring Feature 3's accuracy where source metadata is available, as genuinely necessary rather than a formality. It also suggests that if ARTEMIS ever tried to have the AI check its own citations by just asking the model "is this supported by the source," this paper shows that approach isn't reliable even with frontier models. This is a real, current limitation ARTEMIS inherits, not something its narrower scope or workspace design gets around.

Theme: RAG citation accuracy

The central question here is simple: if an AI answer comes with a citation, does that citation actually mean anything? Three separate properties matter, and the papers below don't always use the same words for them, but they're getting at the same three things. Citation correctness is whether the specific source cited actually supports the specific claim it's attached to. Citation completeness is whether every claim that needs support actually has a citation, not just some of them. Citation faithfulness or attribution is a deeper question, whether the model actually generated the claim from the cited source, or generated the claim from its own general knowledge and then attached a plausible-looking citation afterward. Having a citation says nothing on its own about any of these three.

Rashkin, Nikolaev, Lamm, Aroyo, Collins, Das, Petrov, Tomar, Turc, and Reitter, "Measuring Attribution in Natural Language Generation Models," Computational Linguistics, 2023.
https://aclanthology.org/2023.cl-4.2/ · DOI: 10.1162/coli_a_00486

What the paper does. This is the paper that gives the field a real definition for what it means for generated text to be attributable to a source. They call it AIS, Attributable to Identified Sources, and the test they define is intuitive: a statement counts as attributed to a source if a person would agree with "According to this source, this statement is true." They built a two-stage human annotation pipeline to apply this test consistently, and demonstrated it across three different tasks, conversational question answering, summarization, and table-to-text generation.

The gap. This is a definition and a human annotation method, not an automatic system. Somebody still has to sit down and manually judge each statement against AIS. It also predates the current wave of LLMs that generate citations directly as part of their answer, so it wasn't tested against systems like Bing Chat or Perplexity that ship citations to real users.

ARTEMIS. The "According to P, s" test is a genuinely useful lens for what Feature 3's output should be held to, every claim it pulls into a new file should pass that test against the source document it came from. But this paper only gives ARTEMIS a standard to aim for, not a tool to enforce it automatically, and nothing in the proposal currently applies this standard, formally or informally, to Feature 3's output.

Gao, Yen, Yu, and Chen, "Enabling Large Language Models to Generate Text with Citations," EMNLP 2023.
https://aclanthology.org/2023.emnlp-main.398/ · DOI: 10.18653/v1/2023.emnlp-main.398

What the paper does. They built ALCE, the first benchmark specifically for testing how well LLMs generate text with citations end to end, meaning the system both retrieves supporting evidence and writes an answer that cites it. They built automatic metrics for three separate things, fluency, correctness, and citation quality, and checked that those automatic scores actually line up with what human judges think.

What they found. Current systems have a lot of room to improve. On the ELI5 dataset specifically, even the best-performing models failed to provide complete citation support for their claims about half the time.

The gap. This paper is squarely about citation completeness, not every claim gets backed by a citation, even from the strongest systems tested. It's also a research benchmark meant for testing and comparing systems, not a deployed product, so it tells you the problem is unsolved in research settings but doesn't tell you what happens once a system like this actually ships to real users.

ARTEMIS. Feature 3 works from a small, explicitly user-picked set of documents rather than an open retrieval corpus, which sidesteps part of what ALCE tests, since ARTEMIS never has to decide which documents to retrieve in the first place, the user already chose them. But that only removes the retrieval half of the problem. ALCE's 50% incomplete-citation finding is about the generation half, whether the model actually attaches a citation to every claim it makes, and a smaller, pre-selected document set doesn't automatically fix that. Nothing in the proposal describes a completeness check for Feature 3's output, so this stays an open gap, not something the workspace scope resolves on its own.

Liu, Zhang, and Liang, "Evaluating Verifiability in Generative Search Engines," Findings of the Association for Computational Linguistics: EMNLP 2023. https://aclanthology.org/2023.findings-emnlp.467/ · DOI: 10.18653/v1/2023.findings-emnlp.467

What the paper does. Instead of testing a benchmark, they audited four real, deployed generative search products people were actually using at the time, Bing Chat, NeevaAI, perplexity.ai, and YouChat. They had human annotators check two separate things for each product's answers: citation recall, meaning do all the statements in the answer actually have supporting citations, and citation precision, meaning does each individual citation actually support the specific statement it's attached to.

What they found. On average, only 51.5% of generated sentences were fully supported by citations, and only 74.5% of citations actually supported the statement they were attached to.

The gap. This is the strongest evidence in this theme that citations don't automatically mean trustworthy, because these are real commercial products with a polished, authoritative-looking citation interface, and roughly a quarter of their citations don't hold up and about half their sentences aren't fully backed. The paper is diagnostic only, it audits and reports, it doesn't build or propose a fix, and it doesn't determine whether the failures come from bad retrieval, bad generation, or citations being attached after the answer was already written.

ARTEMIS. This paper is the clearest real-world comparison point for Feature 3, since it shows that even mature, widely used products with dedicated engineering behind their citation systems still fail this test often. That's a reason to expect Feature 3 needs deliberate verification, not an assumption that citing from a small, known set of documents is inherently safer in the way that matters here, precision and completeness. To be direct about where ARTEMIS stands: the proposal narrows the retrieval problem by having the user pre-select documents, but it does not currently include anything resembling the recall or precision checks this paper used to audit real products, so this is an unaddressed gap in the current design, not a partially solved one.

Sources

1. Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. ACL 2020, pages 1906–1919. https://aclanthology.org/2020.acl-main.173/ · DOI: 10.18653/v1/2020.acl-main.173
2. Kryscinski, W., McCann, B., Xiong, C., & Socher, R. (2020). Evaluating the Factual Consistency of Abstractive Text Summarization. EMNLP 2020, pages 9332–9346. https://aclanthology.org/2020.emnlp-main.750/ · DOI: 10.18653/v1/2020.emnlp-main.750
3. Pagnoni, A., Balachandran, V., & Tsvetkov, Y. (2021). Understanding Factuality in Abstractive Summarization with FRANK: A Benchmark for Factuality Metrics. NAACL-HLT 2021, pages 4812–4829. https://aclanthology.org/2021.naacl-main.383/ · DOI: 10.18653/v1/2021.naacl-main.383
4. Laban, P., Kryscinski, W., Agarwal, D., Fabbri, A., Xiong, C., Joty, S., & Wu, C.-S. (2023). SummEdits: Measuring LLM Ability at Factual Reasoning Through The Lens of Summarization. EMNLP 2023, pages 9662–9676. https://aclanthology.org/2023.emnlp-main.600/ · DOI: 10.18653/v1/2023.emnlp-main.600
5. Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., & Reitter, D. (2023). Measuring Attribution in Natural

Language Generation Models. Computational Linguistics, 49(4), 777–840. https://aclanthology.org/2023.cl-4.2/ · DOI: 10.1162/coli_a_00486

6. Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling Large Language Models to Generate Text with Citations. EMNLP 2023, pages 6465–6488. https://aclanthology.org/2023.emnlp-main.398/ · DOI: 10.18653/v1/2023.emnlp-main.398
7. Liu, N. F., Zhang, T., & Liang, P. (2023). Evaluating Verifiability in Generative Search Engines. Findings of the ACL: EMNLP 2023, pages 7001–7025. https://aclanthology.org/2023.findings-emnlp.467/ · DOI: 10.18653/v1/2023.findings-emnlp.467